

isaacsim.sensors

Contents

- Sensor Base
- USD Camera
- Tile-Rendered USD Camera
- Contact Sensor
- Frame Transformer
- Ray-Cast Sensor
- Ray-Cast Camera
- Multi-Mesh Ray-Cast Sensor
- Multi-Mesh Ray-Cast Camera
- Inertia Measurement Unit

Sub-package containing various sensor classes implementations.

This subpackage contains the sensor classes that are compatible with Isaac Sim. We include both USD-based and custom sensors:

- **USD-prim sensors:** Available in Omniverse and require creating a USD prim for them. For instance, RTX ray tracing camera and lidar sensors.
- **USD-schema sensors:** Available in Omniverse and require creating a USD schema on an existing prim. For instance, contact sensors and frame transformers.
- **Custom sensors:** Implemented in Python and do not require creating any USD prim or schema. For instance, warp-based ray-casters.

Due to the above categorization, the `prim` [Back to top](#) used to the sensor's configuration class are interpreted differently based on the sensor type. The following table summarizes the interpretation of the prim paths for different sensor types:

Sensor Type	Example Prim Path	Pre-check
Camera	/World/robot/base/camera	Leaf is available, and it will spawn a USD camera
Contact Sensor	/World/robot/feet_*	Leaf is available and checks if the schema exists
Ray Caster	/World/robot/base	Leaf exists and is a physics body (Articulation / Rigid Body)
Frame Transformer	/World/robot/base	Leaf exists and is a physics body (Articulation / Rigid Body)
Imu	/World/robot/base	Leaf exists and is a physics body (Rigid Body)

Submodules

patterns	Sub-module for ray-casting patterns used by the ray-caster.
-----------------	---

Classes

[Back to top](#)

Print to PDF ►

<code>SensorBase</code>	The base class for implementing a sensor.
<code>SensorBaseCfg</code>	Configuration parameters for a sensor.
<code>Camera</code>	The camera sensor for acquiring visual data.
<code>CameraData</code>	Data container for the camera sensor.
<code>CameraCfg</code>	Configuration for a camera sensor.
<code>TiledCamera</code>	The tiled rendering based camera sensor for acquiring the same data as the Camera class.
<code>TiledCameraCfg</code>	Configuration for a tiled rendering-based camera sensor.
<code>ContactSensor</code>	A contact reporting sensor.
<code>ContactSensorData</code>	Data container for the contact reporting sensor.
<code>ContactSensorCfg</code>	Configuration for the contact sensor.
<code>FrameTransformer</code>	A sensor for reporting frame transforms.
<code>FrameTransformerData</code>	Data container for the frame transformer sensor.
<code>FrameTransformerCfg</code>	Configuration for the frame transformer sensor.
<code>RayCaster</code>	A ray-casting sensor.
<code>RayCasterData</code>	Data container for the ray-cast sensor.
<code>RayCasterCfg</code>	Configuration for the ray-cast sensor.
<code>RayCasterCamera</code>	A ray-casting camera sensor.
<code>RayCasterCameraCfg</code>	Configuration for the ray-cast sensor.
<code>MultiMeshRayCaster</code>	A multi-mesh ray-casting sensor.
<code>MultiMeshRayCasterData</code>	Data container for the multi-mesh ray-cast sensor. Back to top
<code>MultiMeshRayCasterCfg</code>	Configuration for the multi-mesh ray-cast sensor.
<code>MultiMeshRayCasterCamera</code>	A multi-mesh ray-casting camera sensor.

<code>MultiMeshRayCasterCameraCfg</code>	Configuration for the multi-mesh ray-cast camera sensor.
<code>Imu</code>	The Inertia Measurement Unit (IMU) sensor.
<code>ImuCfg</code>	Configuration for an Inertial Measurement Unit (IMU) sensor.

Sensor Base

`class isaacsim.sensors.SensorBase` [\[source\]](#)

The base class for implementing a sensor.

The implementation is based on lazy evaluation. The sensor data is only updated when the user tries accessing the data through the `data` property or sets `force_compute=True` in the `update()` method. This is done to avoid unnecessary computation when the sensor data is not used.

The sensor is updated at the specified update period. If the update period is zero, then the sensor is updated at every simulation step.

Methods:

<code>__init__</code> (cfg)	Initialize the sensor class.
<code>set_debug_vis</code> (debug_vis)	Sets whether to visualize the sensor data.
<code>reset</code> ([env_ids])	Resets the sensor internals.

Attributes:

[Back to top](#)

<code>is_initialized</code>	Whether the sensor is initialized.
<code>num_instances</code>	Number of instances of the sensor.
<code>device</code>	Memory device for computation.
<code>data</code>	Data from the sensor.
<code>has_debug_vis_implementation</code>	Whether the sensor has a debug visualization implemented.

`__init__(cfg: SensorBaseCfg)` [\[source\]](#)

Initialize the sensor class.

Parameters:

cfg – The configuration parameters for the sensor.

property `is_initialized: bool`

Whether the sensor is initialized.

Returns True if the sensor is initialized, False otherwise.

property `num_instances: int`

Number of instances of the sensor.

This is equal to the number of sensors per environment multiplied by the number of environments.

property `device: str`

Memory device for computation.

abstract property `data: ...` [Back to top](#)

Data from the sensor.

This property is only updated when the user tries to access the data. This is done to

avoid unnecessary computation when the sensor data is not used.

For updating the sensor when this property is accessed, you can use the following code snippet in your sensor implementation:

```
# update sensors if needed
self._update_outdated_buffers()
# return the data (where `_data` is the data for the sensor)
return self._data
```



property `has_debug_vis_implementation: bool`

Whether the sensor has a debug visualization implemented.

set_debug_vis(`debug_vis: bool`) → `bool`

[\[source\]](#)

Sets whether to visualize the sensor data.

Parameters:

`debug_vis` – Whether to visualize the sensor data.

Returns:

Whether the debug visualization was successfully set. False if the sensor does not support debug visualization.

reset(`env_ids: Sequence[int] | None = None`)

[\[source\]](#)

Resets the sensor internals.

Parameters:

`env_ids` – The sensor ids to reset. Defaults to None.

class `isaacsim.sensors.SensorBaseCfg`

[\[source\]](#)

Configuration parameters for a s

[Back to top](#)

Attributes:

<code>prim_path</code>	Prim path (or expression) to the sensor.
<code>update_period</code>	Update period of the sensor buffers (in seconds).
<code>history_length</code>	Number of past frames to store in the sensor buffers.
<code>debug_vis</code>	Whether to visualize the sensor.

`prim_path: str`

Prim path (or expression) to the sensor.

Note

The expression can contain the environment namespace regex

`{ENV_REGEX_NS}` which will be replaced with the environment namespace.

Example: `{ENV_REGEX_NS}/Robot/sensor` will be replaced with

`/World/envs/env_.* /Robot/sensor`.

`update_period: float`

Update period of the sensor buffers (in seconds). Defaults to 0.0 (update every step).

`history_length: int`

Number of past frames to store in the sensor buffers. Defaults to 0, which means that only the current data is stored (no history).

`debug_vis: bool`

Whether to visualize the sensor. Defaults to False.

USD Camera

[Back to top](#)

`class isaacsim.sensors.Camera`

[\[source\]](#)

Bases: `SensorBase`

The camera sensor for acquiring visual data.

This class wraps over the [UsdGeom Camera](#) for providing a consistent API for acquiring visual data. It ensures that the camera follows the ROS convention for the coordinate system.

Summarizing from the [replicator extension](#), the following sensor types are supported:

- `"rgb"`: A 3-channel rendered color image.
- `"rgba"`: A 4-channel rendered color image with alpha channel.
- `"distance_to_camera"`: An image containing the distance to camera optical center.
- `"distance_to_image_plane"`: An image containing distances of 3D points from camera plane along camera's z-axis.
- `"depth"`: The same as `"distance_to_image_plane"`.
- `"normals"`: An image containing the local surface normal vectors at each pixel.
- `"motion_vectors"`: An image containing the motion vector data at each pixel.
- `"semantic_segmentation"`: The semantic segmentation data.
- `"instance_segmentation_fast"`: The instance segmentation data.
- `"instance_id_segmentation_fast"`: The instance id segmentation data.

[Back to top](#)

Note

Currently the following sensor types are not supported in a “view” format:

- `"instance_segmentation"`: The instance segmentation data. Please use the fast counterparts instead.
- `"instance_id_segmentation"`: The instance id segmentation data. Please use the fast counterparts instead.
- `"bounding_box_2d_tight"`: The tight 2D bounding box data (only contains non-occluded regions).
- `"bounding_box_2d_tight_fast"`: The tight 2D bounding box data (only contains non-occluded regions).
- `"bounding_box_2d_loose"`: The loose 2D bounding box data (contains occluded regions).
- `"bounding_box_2d_loose_fast"`: The loose 2D bounding box data (contains occluded regions).
- `"bounding_box_3d"`: The 3D view space bounding box data.
- `"bounding_box_3d_fast"`: The 3D view space bounding box data.

Attributes:

[Back to top](#)

<code>cfg</code>	The configuration parameters.
<code>UNSUPPORTED_TYPES</code>	The set of sensor types that are not supported by the camera class.
<code>num_instances</code>	Number of instances of the sensor.
<code>data</code>	Data from the sensor.
<code>frame</code>	Frame number when the measurement took place.
<code>render_product_paths</code>	The path of the render products for the cameras.
<code>image_shape</code>	A tuple containing (height, width) of the camera sensor.
<code>device</code>	Memory device for computation.
<code>has_debug_vis_implementation</code>	Whether the sensor has a debug visualization implemented.
<code>is_initialized</code>	Whether the sensor is initialized.

Methods:

<code>__init__</code> (cfg)	Initializes the camera sensor.
<code>set_intrinsic_matrices</code> (matrices[, ...])	Set parameters of the USD camera from its intrinsic matrix.
<code>set_world_poses</code> ([positions, orientations, ...])	Set the pose of the camera w.r.t.
<code>set_world_poses_from_view</code> (eyes, targets[, ...])	Set the poses of the camera from the eye position and look-at target position.
<code>reset</code> ([env_ids])	Resets the sensor internals.
<code>set_debug_vis</code> (debug_vis)	Sets whether to visualize the sensor data.

[Back to top](#)
cfg: *CameraCfg*

The configuration parameters.

```
UNSUPPORTED_TYPES: set[str] = {'bounding_box_2d_loose',  
'bounding_box_2d_loose_fast', 'bounding_box_2d_tight',  
'bounding_box_2d_tight_fast', 'bounding_box_3d',  
'bounding_box_3d_fast', 'instance_id_segmentation',  
'instance_segmentation'}
```

The set of sensor types that are not supported by the camera class.

`__init__`(*cfg*: *CameraCfg*) [\[source\]](#)

Initializes the camera sensor.

Parameters:

cfg – The configuration parameters.

Raises:

- **RuntimeError** – If no camera prim is found at the given path.
- **ValueError** – If the provided data types are not supported by the camera.

property *num_instances*: *int*

Number of instances of the sensor.

This is equal to the number of sensors per environment multiplied by the number of environments.

property *data*: *CameraData*

Data from the sensor.

This property is only updated when the user tries to access the data. This is done to avoid unnecessary computation when the sensor data is not used.

[Back to top](#)

For updating the sensor when this property is accessed, you can use the following code snippet in your sensor implementation:

```
# update sensors if needed
self._update_outdated_buffers()
# return the data (where `_data` is the data for the sensor)
return self._data
```



property `frame: torch.tensor`

Frame number when the measurement took place.

property `render_product_paths: list[str]`

The path of the render products for the cameras.

This can be used via replicator interfaces to attach to writes or external annotator registry.

property `image_shape: tuple[int, int]`

A tuple containing (height, width) of the camera sensor.

set_intrinsic_matrices(`matrices: torch.Tensor, focal_length: float`
`| None = None, env_ids: Sequence[int] | None = None`) [\[source\]](#)

Set parameters of the USD camera from its intrinsic matrix.

The intrinsic matrix is used to set the following parameters to the USD camera:

- `focal_length`: The focal length of the camera.
- `horizontal_aperture`: The horizontal aperture of the camera.
- `vertical_aperture`: The vertical aperture of the camera.
- `horizontal_aperture_offset`: The horizontal offset of the camera.
- `vertical_aperture_offset`: The vertical offset of the camera.

[Back to top](#)

Warning

Due to limitations of Omniverse camera, we need to assume that the camera is a spherical lens, i.e. has square pixels, and the optical center is centered at the camera eye. If this assumption is not true in the input intrinsic matrix, then the camera will not set up correctly.

Parameters:

- **matrices** – The intrinsic matrices for the camera. Shape is (N, 3, 3).
- **focal_length** – Perspective focal length (in cm) used to calculate pixel size. Defaults to None. If None, focal_length will be calculated $1 / \text{width}$.
- **env_ids** – A sensor ids to manipulate. Defaults to None, which means all sensor indices.

```
set_world_poses(positions: torch.Tensor | None = None,
orientations: torch.Tensor | None = None, env_ids: Sequence[int] |
None = None, convention: Literal['opengl', 'ros', 'world'] = 'ros')
```

Set the pose of the camera w.r.t. the world frame using specified convention. [\[source\]](#)

Since different fields use different conventions for camera orientations, the method allows users to set the camera poses in the specified convention. Possible conventions are:

- **"opengl"** - forward axis: -Z - up axis +Y - Offset is applied in the OpenGL (Usd.Camera) convention
- **"ros"** - forward axis: +Z - up axis -Y - Offset is applied in the ROS convention
- **"world"** - forward axis: +X - up axis +Z - Offset is applied in the World Frame convention

See

```
isaacsim.sensors.camera.utils.convert_camera_frame_orientation_convention()
```

for more details on the conventions.

[Back to top](#)

Parameters:

- **positions** – The cartesian coordinates (in meters). Shape is (N, 3). Defaults to None, in which case the camera position is not changed.
- **orientations** – The quaternion orientation in (w, x, y, z). Shape is (N, 4). Defaults to None, in which case the camera orientation is not changed.
- **env_ids** – A sensor ids to manipulate. Defaults to None, which means all sensor indices.
- **convention** – The convention in which the poses are fed. Defaults to “ros”.

Raises:

RuntimeError – If the camera prim is not set. Need to call `initialize()` method first.

set_world_poses_from_view(eyes: *torch.Tensor*, targets: *torch.Tensor*, env_ids: *Sequence[int] | None = None*) [source]

Set the poses of the camera from the eye position and look-at target position.

Parameters:

- **eyes** – The positions of the camera’s eye. Shape is (N, 3).
- **targets** – The target locations to look at. Shape is (N, 3).
- **env_ids** – A sensor ids to manipulate. Defaults to None, which means all sensor indices.

Raises:

- **RuntimeError** – If the camera prim is not set. Need to call `initialize()` method first.
- **NotImplementedError** – If the stage up-axis is not “Y” or “Z”.

reset(env_ids: *Sequence[int] | None = None*) [source]

Resets the sensor internals.

Parameters:

env_ids – The sensor `Back to top` defaults to None.

property device: *str*

Memory device for computation.

property `has_debug_vis_implementation: bool`

Whether the sensor has a debug visualization implemented.

property `is_initialized: bool`

Whether the sensor is initialized.

Returns True if the sensor is initialized, False otherwise.

set_debug_vis(`debug_vis: bool`) → `bool`

Sets whether to visualize the sensor data.

Parameters:

`debug_vis` – Whether to visualize the sensor data.

Returns:

Whether the debug visualization was successfully set. False if the sensor does not support debug visualization.

class `isaacsim.sensors.CameraData`

[\[source\]](#)

Data container for the camera sensor.

Attributes:

[Back to top](#)

<code>pos_w</code>	Position of the sensor origin in world frame, following ROS convention.
<code>quat_w_world</code>	Quaternion orientation (w, x, y, z) of the sensor origin in world frame, following the world coordinate frame
<code>image_shape</code>	A tuple containing (height, width) of the camera sensor.
<code>intrinsic_matrices</code>	The intrinsic matrices for the camera.
<code>output</code>	The retrieved sensor data with sensor types as key.
<code>info</code>	The retrieved sensor info with sensor types as key.
<code>quat_w_ros</code>	Quaternion orientation (w, x, y, z) of the sensor origin in the world frame, following ROS convention.
<code>quat_w_opengl</code>	Quaternion orientation (w, x, y, z) of the sensor origin in the world frame, following OpenGL / USD Camera convention.

`pos_w`: *torch.Tensor* = None

Position of the sensor origin in world frame, following ROS convention.

Shape is (N, 3) where N is the number of sensors.

`quat_w_world`: *torch.Tensor* = None

Quaternion orientation (w, x, y, z) of the sensor origin in world frame, following the world coordinate frame

Note

World frame convention follows the camera aligned with forward axis +X and up axis +Z.

Shape is (N, 4) where N is the number of sensors. [Back to top](#)

`image_shape`: *tuple[int, int]* = None

A tuple containing (height, width) of the camera sensor.

`intrinsic_matrices: torch.Tensor = None`

The intrinsic matrices for the camera.

Shape is (N, 3, 3) where N is the number of sensors.

`output: dict[str, torch.Tensor] = None`

The retrieved sensor data with sensor types as key.

The format of the data is available in the [Replicator Documentation](#). For semantic-based data, this corresponds to the `"data"` key in the output of the sensor.

`info: list[dict[str, Any]] = None`

The retrieved sensor info with sensor types as key.

This contains extra information provided by the sensor such as semantic segmentation label mapping, prim paths. For semantic-based data, this corresponds to the `"info"` key in the output of the sensor. For other sensor types, the info is empty.

`property quat_w_ros: torch.Tensor`

Quaternion orientation (w, x, y, z) of the sensor origin in the world frame, following ROS convention.

Note

ROS convention follows the camera aligned with forward axis +Z and up axis -Y.

Shape is (N, 4) where N is the number of sensors.

`property quat_w_opengl: torch.Tensor`

[Back to top](#)

Quaternion orientation (w, x, y, z) of the sensor origin in the world frame, following OpenGL / USD Camera convention.

Note

OpenGL convention follows the camera aligned with forward axis -Z and up axis +Y.

Shape is (N, 4) where N is the number of sensors.

`class isaacsim.sensors.CameraCfg`

[\[source\]](#)

Bases: `SensorBaseCfg`

Configuration for a camera sensor.

Attributes:

[Back to top](#)

<code>offset</code>	The offset pose of the sensor's frame from the sensor's parent frame.
<code>spawn</code>	Spawn configuration for the asset.
<code>depth_clipping_behavior</code>	Clipping behavior for the camera for values exceed the maximum value.
<code>data_types</code>	List of sensor names/types to enable for the camera.
<code>prim_path</code>	Prim path (or expression) to the sensor.
<code>update_period</code>	Update period of the sensor buffers (in seconds).
<code>history_length</code>	Number of past frames to store in the sensor buffers.
<code>debug_vis</code>	Whether to visualize the sensor.
<code>width</code>	Width of the image in pixels.
<code>height</code>	Height of the image in pixels.
<code>update_latest_camera_pose</code>	Whether to update the latest camera pose when fetching the camera's data.
<code>semantic_filter</code>	A string or a list specifying a semantic filter predicate.
<code>colorize_semantic_segmentation</code>	Whether to colorize the semantic segmentation images.
<code>colorize_instance_id_segmentation</code>	Whether to colorize the instance ID segmentation images.
<code>colorize_instance_segmentation</code>	Whether to colorize the instance ID segmentation images.
<code>semantic_segmentation_mapping</code>	Dictionary mapping semantics to specific colours

[Back to top](#)

offset: *OffsetCfg*

The offset pose of the sensor's frame from the sensor's parent frame. Defaults to identity.

Note

The parent frame is the frame the sensor attaches to. For example, the parent frame of a camera at path `/World/envs/env_0/Robot/Camera` is `/World/envs/env_0/Robot`.

spawn: *PinholeCameraCfg* | *FisheyeCameraCfg* | *None*

Spawn configuration for the asset.

If *None*, then the prim is not spawned by the asset. Instead, it is assumed that the asset is already present in the scene.

depth_clipping_behavior: *Literal*['max', 'zero', 'none']

Clipping behavior for the camera for values exceed the maximum value. Defaults to "none".

- `"max"`: Values are clipped to the maximum value.
- `"zero"`: Values are clipped to zero.
- `"none"`: No clipping is applied. Values will be returned as `inf`.

data_types: *list*[*str*]

List of sensor names/types to enable for the camera. Defaults to ["rgb"].

Please refer to the `Camera` class for a list of available data types.

prim_path: *str*

[Back to top](#)

Prim path (or expression) to the sensor.

Note

The expression can contain the environment namespace regex

`{ENV_REGEX_NS}` which will be replaced with the environment namespace.

Example: `{ENV_REGEX_NS}/Robot/sensor` will be replaced with

`/World/envs/env_*/Robot/sensor`.

update_period: float

Update period of the sensor buffers (in seconds). Defaults to 0.0 (update every step).

history_length: int

Number of past frames to store in the sensor buffers. Defaults to 0, which means that only the current data is stored (no history).

debug_vis: bool

Whether to visualize the sensor. Defaults to False.

width: int

Width of the image in pixels.

height: int

Height of the image in pixels.

update_latest_camera_pose: bool

Whether to update the latest camera pose when fetching the camera's data. Defaults to False.

[Back to top](#)

If True, the latest camera pose is updated in the camera's data which will slow down performance due to the use of `XformPrimView`. If False, the pose of the camera during initialization is returned.

semantic_filter: str | list[str]

A string or a list specifying a semantic filter predicate. Defaults to `"* : *"`.

If a string, it should be a disjunctive normal form of (semantic type, labels). For examples:

- `"typeA : labelA & !labelB | labelC , typeB: labelA ; typeC: labelE"`: All prims with semantic type “typeA” and label “labelA” but not “labelB” or with label “labelC”. Also, all prims with semantic type “typeB” and label “labelA”, or with semantic type “typeC” and label “labelE”.
- `"typeA : * ; * : labelA"`: All prims with semantic type “typeA” or with label “labelA”

If a list of strings, each string should be a semantic type. The segmentation for prims with semantics of the specified types will be retrieved. For example, if the list is [“class”], only the segmentation for prims with semantics of type “class” will be retrieved.

 See also

For more information on the semantics filter, see the documentation on [Replicator Semantics Schema Editor](#).

colorize_semantic_segmentation: bool

Whether to colorize the semantic segmentation images. Defaults to True.

If True, semantic segmentation is converted to an image where semantic IDs are mapped to colors and returned as a `uint8` 4-channel array. If False, the output is returned as a `int32` array.

colorize_instance_id_segmentation: bool

Whether to colorize the instance ID segmentation images. Defaults to True.

If True, instance id segment [Back to top](#) ted to an image where instance IDs are mapped to colors. and returned as a `uint8` 4-channel array. If False, the output is returned as a `int32` array.

colorize_instance_segmentation: bool

Whether to colorize the instance ID segmentation images. Defaults to True.

If True, instance segmentation is converted to an image where instance IDs are mapped to colors. and returned as a `uint8` 4-channel array. If False, the output is returned as a `int32` array.

semantic_segmentation_mapping: dict

Dictionary mapping semantics to specific colours

Eg.

```
{
  "class:cube_1": (255, 36, 66, 255),
  "class:cube_2": (255, 184, 48, 255),
  "class:cube_3": (55, 255, 139, 255),
  "class:table": (255, 237, 218, 255),
  "class:ground": (100, 100, 100, 255),
  "class:robot": (61, 178, 255, 255),
}
```

Tile-Rendered USD Camera

class isaacsim.sensors.TiledCamera

[\[source\]](#)

Bases: `Camera`

The tiled rendering based camera sensor for acquiring the same data as the Camera class.

This class inherits from the `Camera` class but uses the tiled-rendering API to acquire the visual data. Tiled-rendering concatenates the rendered images from multiple cameras into a single image. This allows for rendering multiple cameras in parallel and is useful for rendering large scenes with multiple cameras efficiently.

The following sensor types are supported:

[Back to top](#)

- `"rgb"`: A 3-channel rendered color image.
- `"rgba"`: A 4-channel rendered color image with alpha channel.
- `"distance_to_camera"`: An image containing the distance to camera optical center.

- `"distance_to_image_plane"`: An image containing distances of 3D points from camera plane along camera's z-axis.
- `"depth"`: Alias for `"distance_to_image_plane"`.
- `"normals"`: An image containing the local surface normal vectors at each pixel.
- `"motion_vectors"`: An image containing the motion vector data at each pixel.
- `"semantic_segmentation"`: The semantic segmentation data.
- `"instance_segmentation_fast"`: The instance segmentation data.
- `"instance_id_segmentation_fast"`: The instance id segmentation data.

Note

Currently the following sensor types are not supported in a “view” format:

- `"instance_segmentation"`: The instance segmentation data. Please use the fast counterparts instead.
- `"instance_id_segmentation"`: The instance id segmentation data. Please use the fast counterparts instead.
- `"bounding_box_2d_tight"`: The tight 2D bounding box data (only contains non-occluded regions).
- `"bounding_box_2d_tight_fast"`: The tight 2D bounding box data (only contains non-occluded regions).
- `"bounding_box_2d_loose"`: The loose 2D bounding box data (contains occluded regions).
- `"bounding_box_2d_loose_fast"`: The loose 2D bounding box data (contains occluded regions).
- `"bounding_box_3d"`: The 3D view space bounding box data.
- `"bounding_box_3d_fast"`: The 3D view space bounding box data.

 **Added in version v1.0.0:** This feature is available starting from Isaac Sim 4.2. Before this version, the tiled rendering APIs were not available.

Attributes:

[Back to top](#)

<code>cfg</code>	The configuration parameters.
<code>UNSUPPORTED_TYPES</code>	The set of sensor types that are not supported by the camera class.
<code>data</code>	Data from the sensor.
<code>device</code>	Memory device for computation.
<code>frame</code>	Frame number when the measurement took place.
<code>has_debug_vis_implementation</code>	Whether the sensor has a debug visualization implemented.
<code>image_shape</code>	A tuple containing (height, width) of the camera sensor.
<code>is_initialized</code>	Whether the sensor is initialized.
<code>num_instances</code>	Number of instances of the sensor.
<code>render_product_paths</code>	The path of the render products for the cameras.

Methods:

<code>__init__</code> (cfg)	Initializes the tiled camera sensor.
<code>reset</code> ([env_ids])	Resets the sensor internals.
<code>set_debug_vis</code> (debug_vis)	Sets whether to visualize the sensor data.
<code>set_intrinsic_matrices</code> (matrices[, ...])	Set parameters of the USD camera from its intrinsic matrix.
<code>set_world_poses</code> ([positions, orientations, ...])	Set the pose of the camera w.r.t.
<code>set_world_poses_from_view</code> (eyes, targets[, ...]) Back to top	Set the poses of the camera from the eye position and look-at target position.

cfg: *TiledCameraCfg*

The configuration parameters.

`__init__(cfg: TiledCameraCfg)` [\[source\]](#)

Initializes the tiled camera sensor.

Parameters:

cfg – The configuration parameters.

Raises:

- **`RuntimeError`** – If no camera prim is found at the given path.
- **`ValueError`** – If the provided data types are not supported by the camera.

`reset(env_ids: Sequence[int] | None = None)` [\[source\]](#)

Resets the sensor internals.

Parameters:

env_ids – The sensor ids to reset. Defaults to None.

`UNSUPPORTED_TYPES: set[str] = {'bounding_box_2d_loose', 'bounding_box_2d_loose_fast', 'bounding_box_2d_tight', 'bounding_box_2d_tight_fast', 'bounding_box_3d', 'bounding_box_3d_fast', 'instance_id_segmentation', 'instance_segmentation'}`

The set of sensor types that are not supported by the camera class.

`property data: CameraData`

Data from the sensor.

This property is only updated when the user tries to access the data. This is done to avoid unnecessary computation when the sensor data is not used.

[Back to top](#)

For updating the sensor when this property is accessed, you can use the following code snippet in your sensor implementation:

```
# update sensors if needed
self._update_outdated_buffers()
# return the data (where `_data` is the data for the sensor)
return self._data
```



property **device**: *str*

Memory device for computation.

property **frame**: *torch.tensor*

Frame number when the measurement took place.

property **has_debug_vis_implementation**: *bool*

Whether the sensor has a debug visualization implemented.

property **image_shape**: *tuple[int, int]*

A tuple containing (height, width) of the camera sensor.

property **is_initialized**: *bool*

Whether the sensor is initialized.

Returns True if the sensor is initialized, False otherwise.

property **num_instances**: *int*

Number of instances of the sensor.

This is equal to the number of sensors per environment multiplied by the number of environments.

[Back to top](#)

property **render_product_paths**: *list[str]*

The path of the render products for the cameras.

This can be used via replicator interfaces to attach to writes or external annotator registry.

set_debug_vis(debug_vis: bool) → bool

Sets whether to visualize the sensor data.

Parameters:

debug_vis – Whether to visualize the sensor data.

Returns:

Whether the debug visualization was successfully set. False if the sensor does not support debug visualization.

set_intrinsic_matrices(matrices: torch.Tensor, focal_length: float | None = None, env_ids: Sequence[int] | None = None)

Set parameters of the USD camera from its intrinsic matrix.

The intrinsic matrix is used to set the following parameters to the USD camera:

- `focal_length`: The focal length of the camera.
- `horizontal_aperture`: The horizontal aperture of the camera.
- `vertical_aperture`: The vertical aperture of the camera.
- `horizontal_aperture_offset`: The horizontal offset of the camera.
- `vertical_aperture_offset`: The vertical offset of the camera.

⚠ Warning

Due to limitations of Omniverse camera, we need to assume that the camera is a spherical lens, i.e. has square pixels, and the optical center is centered at the camera eye. If this assumption is not true in the input intrinsic matrix, then the camera will not set up correctly.

[Back to top](#)

Parameters:

- **matrices** – The intrinsic matrices for the camera. Shape is (N, 3, 3).
- **focal_length** – Perspective focal length (in cm) used to calculate pixel size. Defaults to None. If None, focal_length will be calculated 1 / width.
- **env_ids** – A sensor ids to manipulate. Defaults to None, which means all sensor indices.

```
set_world_poses(positions: torch.Tensor | None = None,
orientations: torch.Tensor | None = None, env_ids: Sequence[int] |
None = None, convention: Literal['opengl', 'ros', 'world'] = 'ros')
```

Set the pose of the camera w.r.t. the world frame using specified convention.

Since different fields use different conventions for camera orientations, the method allows users to set the camera poses in the specified convention. Possible conventions are:

- **"opengl"** - forward axis: -Z - up axis +Y - Offset is applied in the OpenGL (Usd.Camera) convention
- **"ros"** - forward axis: +Z - up axis -Y - Offset is applied in the ROS convention
- **"world"** - forward axis: +X - up axis +Z - Offset is applied in the World Frame convention

See

```
isaacsim.sensors.camera.utils.convert_camera_frame_orientation_convention()
```

for more details on the conventions.

Parameters:

- **positions** – The cartesian coordinates (in meters). Shape is (N, 3). Defaults to None, in which case the camera position in not changed.
- **orientations** – The quaternion orientation in (w, x, y, z). Shape is (N, 4). Defaults to None, in which case the camera orientation in not changed.
- **env_ids** – A sensor ids to manipulate. Defaults to None, which means all sensor indices.
- **convention** – The c [Back to top](#) hich the poses are fed. Defaults to "ros".

Raises:

RuntimeError – If the camera prim is not set. Need to call `initialize()` method first.

```
set_world_poses_from_view(eyes: torch.Tensor, targets:
torch.Tensor, env_ids: Sequence[int] | None = None)
```

Set the poses of the camera from the eye position and look-at target position.

Parameters:

- **eyes** – The positions of the camera's eye. Shape is (N, 3).
- **targets** – The target locations to look at. Shape is (N, 3).
- **env_ids** – A sensor ids to manipulate. Defaults to None, which means all sensor indices.

Raises:

- **RuntimeError** – If the camera prim is not set. Need to call `initialize()` method first.
- **NotImplementedError** – If the stage up-axis is not “Y” or “Z”.

```
class isaacsim.sensors.TiledCameraCfg
```

[\[source\]](#)

Bases: `CameraCfg`

Configuration for a tiled rendering-based camera sensor.

Classes:

`OffsetCfg`

The offset pose of the sensor's frame from the sensor's parent frame.

Attributes:

[Back to top](#)

<code>prim_path</code>	Prim path (or expression) to the sensor.
<code>update_period</code>	Update period of the sensor buffers (in seconds).
<code>history_length</code>	Number of past frames to store in the sensor buffers.
<code>debug_vis</code>	Whether to visualize the sensor.
<code>offset</code>	The offset pose of the sensor's frame from the sensor's parent frame.
<code>spawn</code>	Spawn configuration for the asset.
<code>depth_clipping_behavior</code>	Clipping behavior for the camera for values exceed the maximum value.
<code>data_types</code>	List of sensor names/types to enable for the camera.
<code>width</code>	Width of the image in pixels.
<code>height</code>	Height of the image in pixels.
<code>update_latest_camera_pose</code>	Whether to update the latest camera pose when fetching the camera's data.
<code>semantic_filter</code>	A string or a list specifying a semantic filter predicate.
<code>colorize_semantic_segmentation</code>	Whether to colorize the semantic segmentation images.
<code>colorize_instance_id_segmentation</code>	Whether to colorize the instance ID segmentation images.
<code>colorize_instance_segmentation</code>	Whether to colorize the instance ID segmentation images.
<code>semantic_segmentation_mapping</code>	Dictionary mapping semantics to specific colours

[Back to top](#)

class OffsetCfg

Bases: `object`

The offset pose of the sensor's frame from the sensor's parent frame.

Attributes:

<code>pos</code>	Translation w.r.t.
<code>rot</code>	Quaternion rotation (w, x, y, z) w.r.t.
<code>convention</code>	The convention in which the frame offset is applied.

pos: `tuple[float, float, float]`

Translation w.r.t. the parent frame. Defaults to (0.0, 0.0, 0.0).

rot: `tuple[float, float, float, float]`

Quaternion rotation (w, x, y, z) w.r.t. the parent frame. Defaults to (1.0, 0.0, 0.0, 0.0).

convention: `Literal['opengl', 'ros', 'world']`

The convention in which the frame offset is applied. Defaults to "ros".

- `"opengl"` - forward axis: `-Z` - up axis: `+Y` - Offset is applied in the OpenGL (Usd.Camera) convention.
- `"ros"` - forward axis: `+Z` - up axis: `-Y` - Offset is applied in the ROS convention.
- `"world"` - forward axis: `+X` - up axis: `+Z` - Offset is applied in the World Frame convention.

prim_path: `str`

Prim path (or expression) to the sensor

[Back to top](#)

Note

The expression can contain the environment namespace regex

`{ENV_REGEX_NS}` which will be replaced with the environment namespace.

Example: `{ENV_REGEX_NS}/Robot/sensor` will be replaced with

`/World/envs/env_*/Robot/sensor`.

update_period: float

Update period of the sensor buffers (in seconds). Defaults to 0.0 (update every step).

history_length: int

Number of past frames to store in the sensor buffers. Defaults to 0, which means that only the current data is stored (no history).

debug_vis: bool

Whether to visualize the sensor. Defaults to False.

offset: OffsetCfg

The offset pose of the sensor's frame from the sensor's parent frame. Defaults to identity.

Note

The parent frame is the frame the sensor attaches to. For example, the parent frame of a camera at path `/World/envs/env_0/Robot/Camera` is

`/World/envs/env_0/Robot`.

spawn: PinholeCameraCfg [Back to top](#) **CameraCfg | None**

Spawn configuration for the asset.

If None, then the prim is not spawned by the asset. Instead, it is assumed that the asset is already present in the scene.

depth_clipping_behavior: *Literal*['max', 'zero', 'none']

Clipping behavior for the camera for values exceed the maximum value. Defaults to "none".

- `"max"`: Values are clipped to the maximum value.
- `"zero"`: Values are clipped to zero.
- `"none"`: No clipping is applied. Values will be returned as `inf`.

data_types: *list*[*str*]

List of sensor names/types to enable for the camera. Defaults to ["rgb"].

Please refer to the `Camera` class for a list of available data types.

width: *int*

Width of the image in pixels.

height: *int*

Height of the image in pixels.

update_latest_camera_pose: *bool*

Whether to update the latest camera pose when fetching the camera's data. Defaults to False.

If True, the latest camera pose is updated in the camera's data which will slow down performance due to the use of `XformPrimView`. If False, the pose of the camera during initialization is returned.

semantic_filter: *str* | *list*[*str*]

A string or a list specifying a `Back to top` predicate. Defaults to `"* : *"`.

If a string, it should be a disjunctive normal form of (semantic type, labels). For examples:

- `"typeA : labelA & !labelB | labelC , typeB: labelA ; typeC: labelE"`: All prims with semantic type “typeA” and label “labelA” but not “labelB” or with label “labelC”. Also, all prims with semantic type “typeB” and label “labelA”, or with semantic type “typeC” and label “labelE”.
- `"typeA : * ; * : labelA"`: All prims with semantic type “typeA” or with label “labelA”

If a list of strings, each string should be a semantic type. The segmentation for prims with semantics of the specified types will be retrieved. For example, if the list is [“class”], only the segmentation for prims with semantics of type “class” will be retrieved.

➡ See also

For more information on the semantics filter, see the documentation on [Replicator Semantics Schema Editor](#).

colorize_semantic_segmentation: *bool*

Whether to colorize the semantic segmentation images. Defaults to True.

If True, semantic segmentation is converted to an image where semantic IDs are mapped to colors and returned as a `uint8` 4-channel array. If False, the output is returned as a `int32` array.

colorize_instance_id_segmentation: *bool*

Whether to colorize the instance ID segmentation images. Defaults to True.

If True, instance id segmentation is converted to an image where instance IDs are mapped to colors. and returned as a `uint8` 4-channel array. If False, the output is returned as a `int32` array.

colorize_instance_segmentation: *bool*

Whether to colorize the instance segmentation images. Defaults to True.

If True, instance segmentation is converted to an image where instance IDs are mapped to colors. and returned as a `uint8` 4-channel array. If False, the output is returned as a `int32` array.

semantic_segmentation_mapping: *dict*

Dictionary mapping semantics to specific colours

Eg.

```
{
  "class:cube_1": (255, 36, 66, 255),
  "class:cube_2": (255, 184, 48, 255),
  "class:cube_3": (55, 255, 139, 255),
  "class:table": (255, 237, 218, 255),
  "class:ground": (100, 100, 100, 255),
  "class:robot": (61, 178, 255, 255),
}
```



Contact Sensor

class `isaacsim.sensors.ContactSensor`

[\[source\]](#)

Bases: `SensorBase`

A contact reporting sensor.

The contact sensor reports the normal contact forces on a rigid body in the world frame. It relies on the [PhysX ContactReporter](#) API to be activated on the rigid bodies.

To enable the contact reporter on a rigid body, please make sure to enable the `isaacsim.sim.spawner.RigidBodySpawnerCfg.activate_contact_sensors` on your asset spawner configuration. This will enable the contact reporter on all the rigid bodies in the asset.

The sensor can be configured to report the contact forces on a set of bodies with a given filter pattern using the `ContactSensorCfg.filter_prim_paths_expr`. This is useful when you want to report the contact forces between the sensor bodies and a specific set of bodies in the scene. The data can be accessed using the `ContactSensorData.force_matrix_w`. Please check the documentation on [RigidContact](#) for more details.

The reporting of the filtered contact forces is only possible as one-to-many. This means that only one sensor body in an environment can be filtered against multiple bodies in that environment. If you need to filter multiple sensor bodies against multiple bodies, you need to create separate sensors for each sensor body.

As an example, suppose you want to report the contact forces for all the feet of a robot against an object exclusively. In that case, setting the `ContactSensorCfg.prim_path` and `ContactSensorCfg.filter_prim_paths_expr` with `{ENV_REGEX_NS}/Robot/.*_FOOT` and `{ENV_REGEX_NS}/Object` respectively will not work. Instead, you need to create a separate sensor for each foot and filter it against the object.

Attributes:

<code>cfg</code>	The configuration parameters.
<code>num_instances</code>	Number of instances of the sensor.
<code>data</code>	Data from the sensor.
<code>num_bodies</code>	Number of bodies with contact sensors attached.
<code>body_names</code>	Ordered names of bodies with contact sensors attached.
<code>body_physx_view</code>	View for the rigid bodies captured (PhysX).
<code>contact_physx_view</code>	Contact reporter view for the bodies (PhysX).
<code>device</code>	Memory device for computation.
<code>has_debug_vis_implementation</code>	Whether the sensor has a debug visualization implemented.
<code>is_initialized</code>	Whether the sensor is initialized.

Methods:

[Back to top](#)

<code>__init__</code> (cfg)	Initializes the contact sensor object.
<code>reset</code> ([env_ids])	Resets the sensor internals.
<code>find_bodies</code> (name_keys[, preserve_order])	Find bodies in the articulation based on the name keys.
<code>compute_first_contact</code> (dt[, abs_tol])	Checks if bodies that have established contact within the last <code>dt</code> seconds.
<code>compute_first_air</code> (dt[, abs_tol])	Checks if bodies that have broken contact within the last <code>dt</code> seconds.
<code>set_debug_vis</code> (debug_vis)	Sets whether to visualize the sensor data.

cfg: *ContactSensorCfg*

The configuration parameters.

`__init__`(cfg: *ContactSensorCfg*)

[\[source\]](#)

Initializes the contact sensor object.

Parameters:

cfg – The configuration parameters.

property **num_instances**: *int*

Number of instances of the sensor.

This is equal to the number of sensors per environment multiplied by the number of environments.

property **data**: *ContactSensorData*

Data from the sensor.

[Back to top](#)

This property is only updated when the user tries to access the data. This is done to avoid unnecessary computation when the sensor data is not used.

For updating the sensor when this property is accessed, you can use the following code

snippet in your sensor implementation:

```
# update sensors if needed
self._update_outdated_buffers()
# return the data (where `_data` is the data for the sensor)
return self._data
```



property `num_bodies: int`

Number of bodies with contact sensors attached.

property `body_names: list[str]`

Ordered names of bodies with contact sensors attached.

property `body_physx_view:`

`omni.physics.tensors.impl.api.RigidBodyView`

View for the rigid bodies captured (PhysX).

Note

Use this view with caution. It requires handling of tensors in a specific way.

property `contact_physx_view:`

`omni.physics.tensors.impl.api.RigidContactView`

Contact reporter view for the bodies (PhysX).

Note

Use this view with caution. It requires handling of tensors in a specific way.

[Back to top](#)

reset(`env_ids: Sequence[int] | None = None`)

[\[source\]](#)

Resets the sensor internals.

Parameters:

env_ids – The sensor ids to reset. Defaults to None.

find_bodies(*name_keys*: *str* | *Sequence*[*str*], *preserve_order*: *bool* = *False*) → *tuple*[*list*[*int*], *list*[*str*]] [\[source\]](#)

Find bodies in the articulation based on the name keys.

Parameters:

- **name_keys** – A regular expression or a list of regular expressions to match the body names.
- **preserve_order** – Whether to preserve the order of the name keys in the output. Defaults to False.

Returns:

A tuple of lists containing the body indices and names.

compute_first_contact(*dt*: *float*, *abs_tol*: *float* = $1e-08$) → *torch.Tensor* [\[source\]](#)

Checks if bodies that have established contact within the last `dt` seconds.

This function checks if the bodies have established contact within the last `dt` seconds by comparing the current contact time with the given time period. If the contact time is less than the given time period, then the bodies are considered to be in contact.

Note

The function assumes that `dt` is a factor of the sensor update time-step. In other words $dt/dt_{sensor} = n$, where n is a natural number. This is always true if the sensor is updated by the physics or the environment stepping time-step and the sensor is read by the environment stepping time-step.

Parameters:

- **dt** – The time period [Back to top](#) tact was established.
- **abs_tol** – The absolute tolerance for the comparison.

Returns:

A boolean tensor indicating the bodies that have established contact within the last `dt` seconds. Shape is (N, B), where N is the number of sensors and B is the number of bodies in each sensor.

Raises:

RuntimeError – If the sensor is not configured to track contact time.

compute_first_air(*dt*: float, *abs_tol*: float = 1e-08) → torch.Tensor [\[source\]](#)

Checks if bodies that have broken contact within the last `dt` seconds.

This function checks if the bodies have broken contact within the last `dt` seconds by comparing the current air time with the given time period. If the air time is less than the given time period, then the bodies are considered to not be in contact.

Note

It assumes that `dt` is a factor of the sensor update time-step. In other words, $dt/dt_{sensor} = n$, where n is a natural number. This is always true if the sensor is updated by the physics or the environment stepping time-step and the sensor is read by the environment stepping time-step.

Parameters:

- **dt** – The time period since the contract is broken.
- **abs_tol** – The absolute tolerance for the comparison.

Returns:

A boolean tensor indicating the bodies that have broken contact within the last `dt` seconds. Shape is (N, B), where N is the number of sensors and B is the number of bodies in each sensor.

Raises:

RuntimeError – If the sensor is not configured to track contact time.

[Back to top](#)

property device: str

Memory device for computation.

property `has_debug_vis_implementation: bool`

Whether the sensor has a debug visualization implemented.

property `is_initialized: bool`

Whether the sensor is initialized.

Returns True if the sensor is initialized, False otherwise.

set_debug_vis(`debug_vis: bool`) → `bool`

Sets whether to visualize the sensor data.

Parameters:

`debug_vis` – Whether to visualize the sensor data.

Returns:

Whether the debug visualization was successfully set. False if the sensor does not support debug visualization.

class `isaacsim.sensors.ContactSensorData`

[\[source\]](#)

Data container for the contact reporting sensor.

Attributes:

[Back to top](#)

<code>pos_w</code>	Position of the sensor origin in world frame.
<code>contact_pos_w</code>	Average of the positions of contact points between sensor body and filter prim in world frame.
<code>friction_forces_w</code>	Sum of the friction forces between sensor body and filter prim in world frame.
<code>quat_w</code>	Orientation of the sensor origin in quaternion (w, x, y, z) in world frame.
<code>net_forces_w</code>	The net normal contact forces in world frame.
<code>net_forces_w_history</code>	The net normal contact forces in world frame.
<code>force_matrix_w</code>	The normal contact forces filtered between the sensor bodies and filtered bodies in world frame.
<code>force_matrix_w_history</code>	The normal contact forces filtered between the sensor bodies and filtered bodies in world frame.
<code>last_air_time</code>	Time spent (in s) in the air before the last contact.
<code>current_air_time</code>	Time spent (in s) in the air since the last detach.
<code>last_contact_time</code>	Time spent (in s) in contact before the last detach.
<code>current_contact_time</code>	Time spent (in s) in contact since the last contact.

`pos_w`: *torch.Tensor* | *None* = *None*

Position of the sensor origin in world frame.

Shape is (N, 3), where N is the number of sensors.

Note

If the `ContactSensorCfg.track_pose` is False, then this quantity is None.

[Back to top](#)

`contact_pos_w`: *torch.Tensor* | *None* = *None*

Average of the positions of contact points between sensor body and filter prim in world

frame.

Shape is (N, B, M, 3), where N is the number of sensors, B is number of bodies in each sensor and M is the number of filtered bodies.

Collision pairs not in contact will result in NaN.

Note

- If the `ContactSensorCfg.track_contact_points` is False, then this quantity is None.
- If the `ContactSensorCfg.track_contact_points` is True, a ValueError will be raised if:
 - If the `ContactSensorCfg.filter_prim_paths_expr` is empty.
 - If the `ContactSensorCfg.max_contact_data_per_prim` is not specified or less than 1. will not be calculated.

friction_forces_w: torch.Tensor | None = None

Sum of the friction forces between sensor body and filter prim in world frame.

Shape is (N, B, M, 3), where N is the number of sensors, B is number of bodies in each sensor and M is the number of filtered bodies.

Collision pairs not in contact will result in NaN.

Note

- If the `ContactSensorCfg.track_friction_forces` is False, then this quantity is None.
- If the `ContactSensorCfg.track_friction_forces` is True, a ValueError will be raised if:
 - The `ContactSensorCfg.filter_prim_paths_expr` is empty.
 - The `ContactSensorCfg.max_contact_data_per_prim` is not specified or less than 1.

quat_w: torch.Tensor | None = None

Orientation of the sensor origin in quaternion (w, x, y, z) in world frame.

Shape is (N, 4), where N is the number of sensors.

Note

If the `ContactSensorCfg.track_pose` is False, then this quantity is None.

net_forces_w: *torch.Tensor* | *None* = *None*

The net normal contact forces in world frame.

Shape is (N, B, 3), where N is the number of sensors and B is the number of bodies in each sensor.

Note

This quantity is the sum of the normal contact forces acting on the sensor bodies. It must not be confused with the total contact forces acting on the sensor bodies (which also includes the tangential forces).

net_forces_w_history: *torch.Tensor* | *None* = *None*

The net normal contact forces in world frame.

Shape is (N, T, B, 3), where N is the number of sensors, T is the configured history length and B is the number of bodies in each sensor.

In the history dimension, the first index is the most recent and the last index is the oldest.

Note

This quantity is the sum of the normal contact forces acting on the sensor bodies. It must not be confused with the total contact forces acting on the sensor bodies (which also includes the tangential forces).

force_matrix_w: *torch.Tensor* | *None* = *None*

The normal contact forces filtered between the sensor bodies and filtered bodies in world frame.

Shape is (N, B, M, 3), where N is the number of sensors, B is number of bodies in each sensor and M is the number of filtered bodies.

Note

If the `ContactSensorCfg.filter_prim_paths_expr` is empty, then this quantity is None.

force_matrix_w_history: *torch.Tensor* | *None* = *None*

The normal contact forces filtered between the sensor bodies and filtered bodies in world frame.

Shape is (N, T, B, M, 3), where N is the number of sensors, T is the configured history length, B is number of bodies in each sensor and M is the number of filtered bodies.

In the history dimension, the first index is the most recent and the last index is the oldest.

Note

If the `ContactSensorCfg.filter_prim_paths_expr` is empty, then this quantity is None.

last_air_time: *torch.Tensor* | *None* = *None*

Time spent (in s) in the air before the last contact.

Shape is (N, B), where N is the number of sensors and B is the number of bodies in each sensor.

[Back to top](#)

Note

If the `ContactSensorCfg.track_air_time` is False, then this quantity is None.

current_air_time: torch.Tensor | None = None

Time spent (in s) in the air since the last detach.

Shape is (N, B), where N is the number of sensors and B is the number of bodies in each sensor.

Note

If the `ContactSensorCfg.track_air_time` is False, then this quantity is None.

last_contact_time: torch.Tensor | None = None

Time spent (in s) in contact before the last detach.

Shape is (N, B), where N is the number of sensors and B is the number of bodies in each sensor.

Note

If the `ContactSensorCfg.track_air_time` is False, then this quantity is None.

current_contact_time: torch.Tensor | None = None

Time spent (in s) in contact since the last contact.

Shape is (N, B), where N is [Back to top](#) sensors and B is the number of bodies in each sensor.

Note

If the `ContactSensorCfg.track_air_time` is False, then this quantity is None.

class `isaacsim.sensors.ContactSensorCfg`

[\[source\]](#)

Bases: `SensorBaseCfg`

Configuration for the contact sensor.

Attributes:

[Back to top](#)

<code>track_pose</code>	Whether to track the pose of the sensor's origin.
<code>track_contact_points</code>	Whether to track the contact point locations.
<code>track_friction_forces</code>	Whether to track the friction forces at the contact points.
<code>max_contact_data_count_per_prim</code>	The maximum number of contacts across all batches of the sensor to keep track of.
<code>prim_path</code>	Prim path (or expression) to the sensor.
<code>update_period</code>	Update period of the sensor buffers (in seconds).
<code>history_length</code>	Number of past frames to store in the sensor buffers.
<code>debug_vis</code>	Whether to visualize the sensor.
<code>track_air_time</code>	Whether to track the air/contact time of the bodies (time between contacts).
<code>force_threshold</code>	The threshold on the norm of the contact force that determines whether two bodies are in collision or not.
<code>filter_prim_paths_expr</code>	The list of primitive paths (or expressions) to filter contacts with.
<code>visualizer_cfg</code>	The configuration object for the visualization markers.

`track_pose: bool`

Whether to track the pose of the sensor's origin. Defaults to False.

`track_contact_points: bool`

[Back to top](#)

Whether to track the contact point locations. Defaults to False.

`track_friction_forces: bool`

Whether to track the friction forces at the contact points. Defaults to False.

max_contact_data_count_per_prim: *int*

The maximum number of contacts across all batches of the sensor to keep track of. Default is 4.

This parameter sets the total maximum counts of the simulation across all bodies and environments. The total number of contacts allowed is $\text{max_contact_data_count_per_prim} \times \text{num_envs} \times \text{num_sensor_bodies}$.

Note

If the environment is very contact rich it is suggested to increase this parameter to avoid out of bounds memory errors and loss of contact data leading to inaccurate measurements.

prim_path: *str*

Prim path (or expression) to the sensor.

Note

The expression can contain the environment namespace regex `{ENV_REGEX_NS}` which will be replaced with the environment namespace.

Example: `{ENV_REGEX_NS}/Robot/sensor` will be replaced with `/World/envs/env_.* /Robot/sensor`.

update_period: *float*

Update period of the sensor buffers (in seconds). Defaults to 0.0 (update every step).

[Back to top](#)

history_length: *int*

Number of past frames to store in the sensor buffers. Defaults to 0, which means that only the current data is stored (no history).

debug_vis: bool

Whether to visualize the sensor. Defaults to False.

track_air_time: bool

Whether to track the air/contact time of the bodies (time between contacts). Defaults to False.

force_threshold: float

The threshold on the norm of the contact force that determines whether two bodies are in collision or not.

This value is only used for tracking the mode duration (the time in contact or in air), if

`track_air_time` is True.

filter_prim_paths_expr: list[str]

The list of primitive paths (or expressions) to filter contacts with. Defaults to an empty list, in which case no filtering is applied.

The contact sensor allows reporting contacts between the primitive specified with `prim_path` and other primitives in the scene. For instance, in a scene containing a robot, a ground plane and an object, you can obtain individual contact reports of the base of the robot with the ground plane and the object.

Note

The expression in the list can contain the environment namespace regex `{ENV_REGEX_NS}` which will be replaced with the environment namespace.

Example: `{ENV_REGEX_NS}/Object` will be replaced with `/World/envs/env_.*Object`.

[Back to top](#)

! Attention

The reporting of filtered contacts only works when the sensor primitive `prim_path` corresponds to a single primitive in that environment. If the sensor primitive corresponds to multiple primitives, the filtering will not work as expected. Please check `ContactSensor` for more details. If `track_contact_points` is true, then `filter_prim_paths_expr` cannot be an empty list!

visualizer_cfg: VisualizationMarkersCfg

The configuration object for the visualization markers. Defaults to `CONTACT_SENSOR_MARKER_CFG`.

i Note

This attribute is only used when debug visualization is enabled.

Frame Transformer

class `isaacsim.sensors.FrameTransformer`

[\[source\]](#)

Bases: `SensorBase`

A sensor for reporting frame transforms.

This class provides an interface for reporting the transform of one or more frames (target frames) with respect to another frame (source frame). The source frame is specified by the user as a prim path (`FrameTransformerCfg.prim_path`) and the target frames are specified by the user as a list of prim paths (`FrameTransformerCfg.target_frames`).

The source frame and target frames are assumed to be rigid bodies. The transform of the target frames with respect to the source frame is computed by first extracting the transform of the source frame and target frame from the physics engine and then computing the relative transform between the two.

Additionally, the user can specify an offset for the source frame and each target frame. This is useful for specifying the transform of the desired frame with respect to the body's center of

mass, for instance.

A common example of using this sensor is to track the position and orientation of the end effector of a robotic manipulator. In this case, the source frame would be the body corresponding to the base frame of the manipulator, and the target frame would be the body corresponding to the end effector. Since the end-effector is typically a fictitious body, the user may need to specify an offset from the end-effector to the body of the manipulator.

Attributes:

<code>cfg</code>	The configuration parameters.
<code>data</code>	Data from the sensor.
<code>num_bodies</code>	Returns the number of target bodies being tracked.
<code>body_names</code>	Returns the names of the target bodies being tracked.
<code>device</code>	Memory device for computation.
<code>has_debug_vis_implementation</code>	Whether the sensor has a debug visualization implemented.
<code>is_initialized</code>	Whether the sensor is initialized.
<code>num_instances</code>	Number of instances of the sensor.

Methods:

<code>__init__</code> (<code>cfg</code>)	Initializes the frame transformer object.
<code>reset</code> (<code>[env_ids]</code>)	Resets the sensor internals.
<code>find_bodies</code> (<code>name_keys[, preserve_order]</code>)	Find bodies in the articulation based on the name keys.
<code>set_debug_vis</code> (<code>debug_vis</code>)	Sets whether to visualize the sensor data.

[Back to top](#)

`cfg`: *FrameTransformerCfg*

The configuration parameters.

`__init__(cfg: FrameTransformerCfg)`

[\[source\]](#)

Initializes the frame transformer object.

Parameters:

cfg – The configuration parameters.

`property data: FrameTransformerData`

Data from the sensor.

This property is only updated when the user tries to access the data. This is done to avoid unnecessary computation when the sensor data is not used.

For updating the sensor when this property is accessed, you can use the following code snippet in your sensor implementation:

```
# update sensors if needed
self._update_outdated_buffers()
# return the data (where `_data` is the data for the sensor)
return self._data
```



`property num_bodies: int`

Returns the number of target bodies being tracked.

Note

This is an alias used for consistency with other sensors. Otherwise, we recommend using `len(data.target_frame_names)` to access the number of target frames.

`property body_names: list[str]`

Returns the names of the target bodies being tracked.

[Back to top](#)

Note

This is an alias used for consistency with other sensors. Otherwise, we recommend using `data.target_frame_names` to access the target frame names.

reset(*env_ids*: *Sequence[int] | None = None*) [source]

Resets the sensor internals.

Parameters:

env_ids – The sensor ids to reset. Defaults to None.

find_bodies(*name_keys*: *str | Sequence[str]*, *preserve_order*: *bool = False*) → *tuple[list[int], list[str]]* [source]

Find bodies in the articulation based on the name keys.

Parameters:

- **name_keys** – A regular expression or a list of regular expressions to match the body names.
- **preserve_order** – Whether to preserve the order of the name keys in the output. Defaults to False.

Returns:

A tuple of lists containing the body indices and names.

property device: *str*

Memory device for computation.

property has_debug_vis_implementation: *bool*

Whether the sensor has a debug visualization implemented.

property is_initialized: *bool*

Whether the sensor is initialized.

Returns True if the sensor is initialized, False otherwise.

property `num_instances: int`

Number of instances of the sensor.

This is equal to the number of sensors per environment multiplied by the number of environments.

set_debug_vis(`debug_vis: bool`) → `bool`

Sets whether to visualize the sensor data.

Parameters:

`debug_vis` – Whether to visualize the sensor data.

Returns:

Whether the debug visualization was successfully set. False if the sensor does not support debug visualization.

class `isaacsim.sensors.FrameTransformerData` [\[source\]](#)

Data container for the frame transformer sensor.

Attributes:

[Back to top](#)

<code>target_frame_names</code>	Target frame names (this denotes the order in which that frame data is ordered).
<code>target_pos_source</code>	Position of the target frame(s) relative to source frame.
<code>target_quat_source</code>	Orientation of the target frame(s) relative to source frame quaternion (w, x, y, z).
<code>target_pos_w</code>	Position of the target frame(s) after offset (in world frame).
<code>target_quat_w</code>	Orientation of the target frame(s) after offset (in world frame) quaternion (w, x, y, z).
<code>source_pos_w</code>	Position of the source frame after offset (in world frame).
<code>source_quat_w</code>	Orientation of the source frame after offset (in world frame) quaternion (w, x, y, z).

`target_frame_names: list[str] = None`

Target frame names (this denotes the order in which that frame data is ordered).

The frame names are resolved from the `FrameTransformerCfg.FrameCfg.name` field. This does not necessarily follow the order in which the frames are defined in the config due to the regex matching.

`target_pos_source: torch.Tensor = None`

Position of the target frame(s) relative to source frame.

Shape is (N, M, 3), where N is the number of environments, and M is the number of target frames.

`target_quat_source: torch.Tensor = None`

Orientation of the target frame(s) relative to source frame quaternion (w, x, y, z).

[Back to top](#)

Shape is (N, M, 4), where N is the number of environments, and M is the number of target frames.

target_pos_w: *torch.Tensor* = *None*

Position of the target frame(s) after offset (in world frame).

Shape is (N, M, 3), where N is the number of environments, and M is the number of target frames.

target_quat_w: *torch.Tensor* = *None*

Orientation of the target frame(s) after offset (in world frame) quaternion (w, x, y, z).

Shape is (N, M, 4), where N is the number of environments, and M is the number of target frames.

source_pos_w: *torch.Tensor* = *None*

Position of the source frame after offset (in world frame).

Shape is (N, 3), where N is the number of environments.

source_quat_w: *torch.Tensor* = *None*

Orientation of the source frame after offset (in world frame) quaternion (w, x, y, z).

Shape is (N, 4), where N is the number of environments.

class isaacsim.sensors.FrameTransformerCfg [\[source\]](#)

Bases: [SensorBaseCfg](#)

Configuration for the frame transformer sensor.

Classes:

FrameCfg	Information specific to a coordinate frame.
--------------------------	---

[Back to top](#)

Attributes:

<code>update_period</code>	Update period of the sensor buffers (in seconds).
<code>history_length</code>	Number of past frames to store in the sensor buffers.
<code>debug_vis</code>	Whether to visualize the sensor.
<code>prim_path</code>	The prim path of the body to transform from (source frame).
<code>source_frame_offset</code>	The pose offset from the source prim frame.
<code>target_frames</code>	A list of the target frames.
<code>visualizer_cfg</code>	The configuration object for the visualization markers.

class `FrameCfg` [\[source\]](#)

Bases: `object`

Information specific to a coordinate frame.

Attributes:

<code>prim_path</code>	The prim path corresponding to a rigid body.
<code>name</code>	User-defined name for the new coordinate frame.
<code>offset</code>	The pose offset from the parent prim frame.

`prim_path: str`

The prim path corresponding to a rigid body.

This can be a regex pattern to match multiple prims. For example, “/Robot/.*” will match all prims under “/Robot”.

This means that if the source `FrameTransformerCfg.prim_path` is “/Robot/base”, and the target `FrameTransformerCfg.FrameCfg.prim_path` is “/Robot/.*”, then the frame transformer will track the poses of all the prims under “/Robot”, including “/Robot/base” (even though the result in an identity pose w.r.t. the source frame).

[Back to top](#)

`name: str | None`

User-defined name for the new coordinate frame. Defaults to None.

If None, then the name is extracted from the leaf of the prim path.

offset: *OffsetCfg*

The pose offset from the parent prim frame.

update_period: *float*

Update period of the sensor buffers (in seconds). Defaults to 0.0 (update every step).

history_length: *int*

Number of past frames to store in the sensor buffers. Defaults to 0, which means that only the current data is stored (no history).

debug_vis: *bool*

Whether to visualize the sensor. Defaults to False.

prim_path: *str*

The prim path of the body to transform from (source frame).

source_frame_offset: *OffsetCfg*

The pose offset from the source prim frame.

target_frames: *list[FrameCfg]*

A list of the target frames.

This allows a single FrameTransformer to handle multiple target prims. For example, in a quadruped, we can use a single FrameTransformer to track each foot's position and orientation in the body frame. [Back to top](#) [Frame offsets](#).

visualizer_cfg: *VisualizationMarkersCfg*

The configuration object for the visualization markers. Defaults to

FRAME_MARKER_CFG.

Note

This attribute is only used when debug visualization is enabled.

class `isaacsim.sensors.OffsetCfg`

[\[source\]](#)

The offset pose of one frame relative to another frame.

Attributes:

<code>pos</code>	Translation w.r.t.
<code>rot</code>	Quaternion rotation (w, x, y, z) w.r.t.

pos: `tuple[float, float, float]`

Translation w.r.t. the parent frame. Defaults to (0.0, 0.0, 0.0).

rot: `tuple[float, float, float, float]`

Quaternion rotation (w, x, y, z) w.r.t. the parent frame. Defaults to (1.0, 0.0, 0.0, 0.0).

Ray-Cast Sensor

class `isaacsim.sensors.RayCaster`

[\[source\]](#)

Bases: `SensorBase`

A ray-casting sensor.

The ray-caster uses a set of rays to detect collisions with meshes in the scene. The rays are defined in the sensor's local coordinate frame. The sensor can be configured to ray-cast against a set of meshes with a given ray pattern.

[Back to top](#)

The meshes are parsed from the list of primitive paths provided in the configuration. These are then converted to warp meshes and stored in the `warp_meshes` list. The ray-caster then ray-casts against these warp meshes using the ray pattern provided in the configuration.

Note

Currently, only static meshes are supported. Extending the warp mesh to support dynamic meshes is a work in progress.

Attributes:

<code>cfg</code>	The configuration parameters.
<code>meshes</code>	A dictionary to store warp meshes for raycasting, shared across all instances.
<code>num_instances</code>	Number of instances of the sensor.
<code>data</code>	Data from the sensor.
<code>device</code>	Memory device for computation.
<code>has_debug_vis_implementation</code>	Whether the sensor has a debug visualization implemented.
<code>is_initialized</code>	Whether the sensor is initialized.

Methods:

<code>__init__</code> (cfg)	Initializes the ray-caster object.
<code>reset</code> ([env_ids])	Resets the sensor internals.
<code>set_debug_vis</code> (debug_vis)	Sets whether to visualize the sensor data.

cfg: *RayCasterCfg*

The configuration parameters.

meshes: `ClassVar[dict[str, WarpMesh]] = {}`

[Back to top](#)

A dictionary to store warp meshes for raycasting, shared across all instances.

The keys correspond to the prim path for the meshes, and values are the corresponding warp Mesh objects.

`__init__(cfg: RayCasterCfg)`[\[source\]](#)

Initializes the ray-caster object.

Parameters:

cfg – The configuration parameters.

property `num_instances: int`

Number of instances of the sensor.

This is equal to the number of sensors per environment multiplied by the number of environments.

property `data: RayCasterData`

Data from the sensor.

This property is only updated when the user tries to access the data. This is done to avoid unnecessary computation when the sensor data is not used.

For updating the sensor when this property is accessed, you can use the following code snippet in your sensor implementation:

```
# update sensors if needed
self._update_outdated_buffers()
# return the data (where `_data` is the data for the sensor)
return self._data
```

**`reset(env_ids: Sequence[int] | None = None)`**[\[source\]](#)

Resets the sensor internals.

Parameters:

env_ids – The sensor ids to reset. Defaults to None.

[Back to top](#)

property `device: str`

Memory device for computation.

property `has_debug_vis_implementation: bool`

Whether the sensor has a debug visualization implemented.

property `is_initialized: bool`

Whether the sensor is initialized.

Returns True if the sensor is initialized, False otherwise.

set_debug_vis(`debug_vis: bool`) → `bool`

Sets whether to visualize the sensor data.

Parameters:

`debug_vis` – Whether to visualize the sensor data.

Returns:

Whether the debug visualization was successfully set. False if the sensor does not support debug visualization.

class `isaacsim.sensors.RaycastData`

[\[source\]](#)

Data container for the ray-cast sensor.

Attributes:

<code>pos_w</code>	Position of the sensor origin in world frame.
<code>quat_w</code>	Orientation of the sensor origin in quaternion (w, x, y, z) in world frame.
<code>ray_hits_w</code>	The ray hit positions in the world frame.

pos_w: torch.Tensor = None

Position of the sensor origin in world frame.

Shape is (N, 3), where N is the number of rays. [Back to top](#) [sensors](#).

quat_w: torch.Tensor = None

Orientation of the sensor origin in quaternion (w, x, y, z) in world frame.

Shape is (N, 4), where N is the number of sensors.

ray_hits_w: *torch.Tensor* = *None*

The ray hit positions in the world frame.

Shape is (N, B, 3), where N is the number of sensors, B is the number of rays in the scan pattern per sensor.

class isaacsim.sensors.**RayCasterCfg**

[\[source\]](#)

Bases: **SensorBaseCfg**

Configuration for the ray-cast sensor.

Classes:

OffsetCfg

The offset pose of the sensor's frame from the sensor's parent frame.

Attributes:

[Back to top](#)

<code>mesh_prim_paths</code>	The list of mesh primitive paths to ray cast against.
<code>offset</code>	The offset pose of the sensor's frame from the sensor's parent frame.
<code>attach_yaw_only</code>	Whether the rays' starting positions and directions only track the yaw orientation.
<code>prim_path</code>	Prim path (or expression) to the sensor.
<code>update_period</code>	Update period of the sensor buffers (in seconds).
<code>history_length</code>	Number of past frames to store in the sensor buffers.
<code>debug_vis</code>	Whether to visualize the sensor.
<code>ray_alignment</code>	Specify in what frame the rays are projected onto the ground.
<code>pattern_cfg</code>	The pattern that defines the local ray starting positions and directions.
<code>max_distance</code>	Maximum distance (in meters) from the sensor to ray cast to.
<code>drift_range</code>	The range of drift (in meters) to add to the ray starting positions (xyz) in world frame.
<code>ray_cast_drift_range</code>	The range of drift (in meters) to add to the projected ray points in local projection frame.
<code>visualizer_cfg</code>	The configuration object for the visualization markers.

class `OffsetCfg`

[\[source\]](#)

Bases: `object`

The offset pose of the sensor's frame from the sensor's parent frame.

Attributes:

Back to top	
<code>pos</code>	Translation w.r.t.
<code>rot</code>	Quaternion rotation (w, x, y, z) w.r.t.

pos: `tuple[float, float, float]`

Translation w.r.t. the parent frame. Defaults to (0.0, 0.0, 0.0).

rot: *tuple[float, float, float, float]*

Quaternion rotation (w, x, y, z) w.r.t. the parent frame. Defaults to (1.0, 0.0, 0.0, 0.0).

mesh_prim_paths: *list[str]*

The list of mesh primitive paths to ray cast against.

Note

Currently, only a single static mesh is supported. We are working on supporting multiple static meshes and dynamic meshes.

offset: *OffsetCfg*

The offset pose of the sensor's frame from the sensor's parent frame. Defaults to identity.

attach_yaw_only: *bool | None*

Whether the rays' starting positions and directions only track the yaw orientation. Defaults to None, which doesn't raise a warning of deprecated usage.

This is useful for ray-casting height maps, where only yaw rotation is needed.

! *Deprecated since version 2.1.1:* This attribute is deprecated and will be removed in the future. Please use `ray_alignment` instead.

To get the same behavior as setting this parameter to `True` or `False`, set `ray_alignment` to `"yaw"` or `"base"` respectively.

[Back to top](#)

prim_path: *str*

Prim path (or expression) to the sensor.

Note

The expression can contain the environment namespace regex

`{ENV_REGEX_NS}` which will be replaced with the environment namespace.

Example: `{ENV_REGEX_NS}/Robot/sensor` will be replaced with

`/World/envs/env_*/Robot/sensor`.

update_period: float

Update period of the sensor buffers (in seconds). Defaults to 0.0 (update every step).

history_length: int

Number of past frames to store in the sensor buffers. Defaults to 0, which means that only the current data is stored (no history).

debug_vis: bool

Whether to visualize the sensor. Defaults to False.

ray_alignment: Literal['base', 'yaw', 'world']

Specify in what frame the rays are projected onto the ground. Default is “base”.

The options are:

- `base` if the rays’ starting positions and directions track the full root position and orientation.
- `yaw` if the rays’ starting positions and directions track root position and only yaw component of the orientation. This is useful for ray-casting height maps.
- `world` if rays’ starting positions and directions are always fixed. This is useful in combination with a mapping package on the robot and querying ray-casts in a global frame.

[Back to top](#)

pattern_cfg: PatternBaseCfg

The pattern that defines the local ray starting positions and directions.

max_distance: *float*

Maximum distance (in meters) from the sensor to ray cast to. Defaults to 1e6.

drift_range: *tuple[float, float]*

The range of drift (in meters) to add to the ray starting positions (xyz) in world frame. Defaults to (0.0, 0.0).

For floating base robots, this is useful for simulating drift in the robot's pose estimation.

ray_cast_drift_range: *dict[str, tuple[float, float]]*

The range of drift (in meters) to add to the projected ray points in local projection frame. Defaults to a dictionary with zero drift for each x, y and z axis.

For floating base robots, this is useful for simulating drift in the robot's pose estimation.

visualizer_cfg: *VisualizationMarkersCfg*

The configuration object for the visualization markers. Defaults to RAY_CASTER_MARKER_CFG.

Note

This attribute is only used when debug visualization is enabled.

Ray-Cast Camera

class `isaacsim.sensors.RayCasterCamera`[\[source\]](#)

Bases: `RayCaster`

A ray-casting camera sensor.

[Back to top](#)

The ray-caster camera uses a set of rays to get the distances to meshes in the scene. The rays are defined in the sensor's local coordinate frame. The sensor has the same interface as the `isaacsim.sensors.Camera` that implements the camera class through USD camera prims. However, this class provides a faster image generation. The sensor converts meshes

from the list of primitive paths provided in the configuration to Warp meshes. The camera then ray-casts against these Warp meshes only.

Currently, only the following annotators are supported:

- `"distance_to_camera"`: An image containing the distance to camera optical center.
- `"distance_to_image_plane"`: An image containing distances of 3D points from camera plane along camera's z-axis.
- `"normals"`: An image containing the local surface normal vectors at each pixel.

Note

Currently, only static meshes are supported. Extending the warp mesh to support dynamic meshes is a work in progress.

Attributes:

<code>cfg</code>	The configuration parameters.
<code>UNSUPPORTED_TYPES</code>	A set of sensor types that are not supported by the ray-caster camera.
<code>data</code>	Data from the sensor.
<code>image_shape</code>	A tuple containing (height, width) of the camera sensor.
<code>frame</code>	Frame number when the measurement took place.
<code>device</code>	Memory device for computation.
<code>has_debug_vis_implementation</code>	Whether the sensor has a debug visualization implemented.
<code>is_initialized</code>	Whether the sensor is initialized.
<code>meshes</code>	Back to top array to store warp meshes for raycasting, shared across all instances.
<code>num_instances</code>	Number of instances of the sensor.

Methods:

<code>__init__</code> (cfg)	Initializes the camera object.
<code>set_intrinsic_matrices</code> (matrices[, ...])	Set the intrinsic matrix of the camera.
<code>reset</code> ([env_ids])	Resets the sensor internals.
<code>set_world_poses</code> ([positions, orientations, ...])	Set the pose of the camera w.r.t.
<code>set_world_poses_from_view</code> (eyes, targets[, ...])	Set the poses of the camera from the eye position and look-at target position.
<code>set_debug_vis</code> (debug_vis)	Sets whether to visualize the sensor data.

cfg: *RayCasterCameraCfg*

The configuration parameters.

UNSUPPORTED_TYPES: *ClassVar[set[str]]* = {'bounding_box_2d_loose', 'bounding_box_2d_loose_fast', 'bounding_box_2d_tight', 'bounding_box_2d_tight_fast', 'bounding_box_3d', 'bounding_box_3d_fast', 'instance_id_segmentation', 'instance_id_segmentation_fast', 'instance_segmentation', 'instance_segmentation_fast', 'motion_vectors', 'rgb', 'semantic_segmentation', 'skeleton_data'}

A set of sensor types that are not supported by the ray-caster camera.

__init__(cfg: *RayCasterCameraCfg*)[\[source\]](#)

Initializes the camera object.

Parameters:[Back to top](#)

cfg – The configuration parameters.

Raises:

ValueError – If the provided data types are not supported by the ray-caster camera.

property data: *CameraData*

Data from the sensor.

This property is only updated when the user tries to access the data. This is done to avoid unnecessary computation when the sensor data is not used.

For updating the sensor when this property is accessed, you can use the following code snippet in your sensor implementation:

```
# update sensors if needed
self._update_outdated_buffers()
# return the data (where `_data` is the data for the sensor)
return self._data
```

**property image_shape: *tuple[int, int]***

A tuple containing (height, width) of the camera sensor.

property frame: *torch.tensor*

Frame number when the measurement took place.

set_intrinsic_matrices(*matrices: torch.Tensor, focal_length: float*
= 1.0, env_ids: Sequence[int] | None = None) [\[source\]](#)

Set the intrinsic matrix of the camera.

Parameters:

- **matrices** – The intrinsic matrices for the camera. Shape is (N, 3, 3).
- **focal_length** – Focal length to use when computing aperture values (in cm). Defaults to 1.0.
- **env_ids** – A sensor ids to manipulate. Defaults to None, which means all sensor indices.

[Back to top](#)

reset(*env_ids: Sequence[int] | None = None*) [\[source\]](#)

Resets the sensor internals.

Parameters:

env_ids – The sensor ids to reset. Defaults to None.

```
set_world_poses(positions: torch.Tensor | None = None,
orientations: torch.Tensor | None = None, env_ids: Sequence[int] |
None = None, convention: Literal['opengl', 'ros', 'world'] = 'ros')
```

Set the pose of the camera w.r.t. the world frame using specified convention. [\[source\]](#)

Since different fields use different conventions for camera orientations, the method allows users to set the camera poses in the specified convention. Possible conventions are:

- **"opengl"** - forward axis: -Z - up axis +Y - Offset is applied in the OpenGL (Usd.Camera) convention
- **"ros"** - forward axis: +Z - up axis -Y - Offset is applied in the ROS convention
- **"world"** - forward axis: +X - up axis +Z - Offset is applied in the World Frame convention

See `isaacsim.utils.maths.convert_camera_frame_orientation_convention()` for more details on the conventions.

Parameters:

- **positions** – The cartesian coordinates (in meters). Shape is (N, 3). Defaults to None, in which case the camera position is not changed.
- **orientations** – The quaternion orientation in (w, x, y, z). Shape is (N, 4). Defaults to None, in which case the camera orientation is not changed.
- **env_ids** – A sensor ids to manipulate. Defaults to None, which means all sensor indices.
- **convention** – The convention in which the poses are fed. Defaults to "ros".

Raises:

RuntimeError – If the camera prim is not set. Need to call `initialize()` method first.

[Back to top](#)

```
set_world_poses_from_view(eyes: torch.Tensor, targets:
torch.Tensor, env_ids: Sequence[int] | None = None) \[source\]
```

Set the poses of the camera from the eye position and look-at target position.

Parameters:

- **eyes** – The positions of the camera's eye. Shape is $N, 3$).
- **targets** – The target locations to look at. Shape is $(N, 3)$.
- **env_ids** – A sensor ids to manipulate. Defaults to None, which means all sensor indices.

Raises:

- **RuntimeError** – If the camera prim is not set. Need to call `initialize()` method first.
- **NotImplementedError** – If the stage up-axis is not “Y” or “Z”.

property device: str

Memory device for computation.

property has_debug_vis_implementation: bool

Whether the sensor has a debug visualization implemented.

property is_initialized: bool

Whether the sensor is initialized.

Returns True if the sensor is initialized, False otherwise.

meshes: ClassVar[dict[str, wp.Mesh]] = {}

A dictionary to store warp meshes for raycasting, shared across all instances.

The keys correspond to the prim path for the meshes, and values are the corresponding warp Mesh objects.

[Back to top](#)

property num_instances: int

Number of instances of the sensor.

This is equal to the number of sensors per environment multiplied by the number of

environments.

`set_debug_vis(debug_vis: bool) → bool`

Sets whether to visualize the sensor data.

Parameters:

`debug_vis` – Whether to visualize the sensor data.

Returns:

Whether the debug visualization was successfully set. False if the sensor does not support debug visualization.

`class isaacsim.sensors.RayCasterCameraCfg` [\[source\]](#)

Bases: `RayCasterCfg`

Configuration for the ray-cast sensor.

Attributes:

[Back to top](#)

<code>offset</code>	The offset pose of the sensor's frame from the sensor's parent frame.
<code>prim_path</code>	Prim path (or expression) to the sensor.
<code>update_period</code>	Update period of the sensor buffers (in seconds).
<code>history_length</code>	Number of past frames to store in the sensor buffers.
<code>debug_vis</code>	Whether to visualize the sensor.
<code>mesh_prim_paths</code>	The list of mesh primitive paths to ray cast against.
<code>attach_yaw_only</code>	Whether the rays' starting positions and directions only track the yaw orientation.
<code>ray_alignment</code>	Specify in what frame the rays are projected onto the ground.
<code>max_distance</code>	Maximum distance (in meters) from the sensor to ray cast to.
<code>drift_range</code>	The range of drift (in meters) to add to the ray starting positions (xyz) in world frame.
<code>ray_cast_drift_range</code>	The range of drift (in meters) to add to the projected ray points in local projection frame.
<code>visualizer_cfg</code>	The configuration object for the visualization markers.
<code>data_types</code>	List of sensor names/types to enable for the camera.
<code>depth_clipping_behavior</code>	Clipping behavior for the camera for values exceed the maximum value.
<code>pattern_cfg</code>	The pattern that defines the local ray starting positions and directions in a pinhole camera pattern.

`offset`: *OffsetCfg*

[Back to top](#)

The offset pose of the sensor's frame from the sensor's parent frame. Defaults to identity.

prim_path: *str*

Prim path (or expression) to the sensor.

Note

The expression can contain the environment namespace regex

`{ENV_REGEX_NS}` which will be replaced with the environment namespace.

Example: `{ENV_REGEX_NS}/Robot/sensor` will be replaced with

`/World/envs/env_.* /Robot/sensor`.

update_period: *float*

Update period of the sensor buffers (in seconds). Defaults to 0.0 (update every step).

history_length: *int*

Number of past frames to store in the sensor buffers. Defaults to 0, which means that only the current data is stored (no history).

debug_vis: *bool*

Whether to visualize the sensor. Defaults to False.

mesh_prim_paths: *list[str]*

The list of mesh primitive paths to ray cast against.

Note

Currently, only a single static mesh is supported. We are working on supporting multiple static meshes and dynamic meshes.

[Back to top](#)

attach_yaw_only: *bool* | *None*

Whether the rays' starting positions and directions only track the yaw orientation.

Defaults to None, which doesn't raise a warning of deprecated usage.

This is useful for ray-casting height maps, where only yaw rotation is needed.

! **Deprecated since version 2.1.1:** This attribute is deprecated and will be removed in the future. Please use `ray_alignment` instead.

To get the same behavior as setting this parameter to `True` or `False`, set `ray_alignment` to `"yaw"` or `"base"` respectively.

ray_alignment: `Literal['base', 'yaw', 'world']`

Specify in what frame the rays are projected onto the ground. Default is “base”.

The options are:

- `base` if the rays’ starting positions and directions track the full root position and orientation.
- `yaw` if the rays’ starting positions and directions track root position and only yaw component of the orientation. This is useful for ray-casting height maps.
- `world` if rays’ starting positions and directions are always fixed. This is useful in combination with a mapping package on the robot and querying ray-casts in a global frame.

max_distance: `float`

Maximum distance (in meters) from the sensor to ray cast to. Defaults to 1e6.

drift_range: `tuple[float, float]`

The range of drift (in meters) to add to the ray starting positions (xyz) in world frame. Defaults to (0.0, 0.0).

For floating base robots, this is useful for simulating drift in the robot’s pose estimation.

[Back to top](#)

ray_cast_drift_range: `dict[str, tuple[float, float]]`

The range of drift (in meters) to add to the projected ray points in local projection frame. Defaults to a dictionary with zero drift for each x, y and z axis.

For floating base robots, this is useful for simulating drift in the robot's pose estimation.

visualizer_cfg: *VisualizationMarkersCfg*

The configuration object for the visualization markers. Defaults to `RAY_CASTER_MARKER_CFG`.

Note

This attribute is only used when debug visualization is enabled.

data_types: *list[str]*

List of sensor names/types to enable for the camera. Defaults to `["distance_to_image_plane"]`.

depth_clipping_behavior: *Literal['max', 'zero', 'none']*

Clipping behavior for the camera for values exceed the maximum value. Defaults to `"none"`.

- `"max"`: Values are clipped to the maximum value.
- `"zero"`: Values are clipped to zero.
- `"none"`: No clipping is applied. Values will be returned as `inf` for `distance_to_camera` and `nan` for `distance_to_image_plane` data type.

pattern_cfg: *PinholeCameraPatternCfg*

The pattern that defines the local ray starting positions and directions in a pinhole camera pattern.

Multi-Mesh Ray-Casting Sensor

[Back to top](#)

class `isaacsim.sensors.MultiMeshRayCaster`

[\[source\]](#)

Bases: `RayCaster`

A multi-mesh ray-casting sensor.

The ray-caster uses a set of rays to detect collisions with meshes in the scene. The rays are defined in the sensor's local coordinate frame. The sensor can be configured to ray-cast against a set of meshes with a given ray pattern.

The meshes are parsed from the list of primitive paths provided in the configuration. These are then converted to warp meshes and stored in the `meshes` list. The ray-caster then ray-casts against these warp meshes using the ray pattern provided in the configuration.

Compared to the default RayCaster, the MultiMeshRayCaster provides additional functionality and flexibility as an extension of the default RayCaster with the following enhancements:

- Raycasting against multiple target types : Supports primitive shapes (spheres, cubes, etc.) as well as arbitrary meshes.
- Dynamic mesh tracking : Keeps track of specified meshes, enabling raycasting against moving parts (e.g., robot links, articulated bodies, or dynamic obstacles).
- Memory-efficient caching : Avoids redundant memory usage by reusing mesh data across environments.

Example usage to raycast against the visual meshes of a robot (e.g. ANYmal):

```
ray_caster_cfg = MultiMeshRayCasterCfg(
    prim_path="{ENV_REGEX_NS}/Robot",
    mesh_prim_paths=[
        "/World/Ground",
        MultiMeshRayCasterCfg.RaycastTargetCfg(prim_expr="{ENV_REGEX_NS}/Robo
        MultiMeshRayCasterCfg.RaycastTargetCfg(prim_expr="{ENV_REGEX_NS}/Robo
        MultiMeshRayCasterCfg.RaycastTargetCfg(prim_expr="{ENV_REGEX_NS}/Robo
        MultiMeshRayCasterCfg.RaycastTargetCfg(prim_expr="{ENV_REGEX_NS}/Robo
        MultiMeshRayCasterCfg.RaycastTargetCfg(prim_expr="{ENV_REGEX_NS}/Robo
    ],
    ray_alignment="world",
    pattern_cfg=patterns.GridPatternCfg(resolution=0.02, size=(2.5, 2.5), dir
)
```

Attributes:

[Back to top](#)

<code>cfg</code>	The configuration parameters.
<code>mesh_views</code>	A dictionary to store mesh views for raycasting, shared across all instances.
<code>data</code>	Data from the sensor.
<code>device</code>	Memory device for computation.
<code>has_debug_vis_implementation</code>	Whether the sensor has a debug visualization implemented.
<code>is_initialized</code>	Whether the sensor is initialized.
<code>meshes</code>	A dictionary to store warp meshes for raycasting, shared across all instances.
<code>num_instances</code>	Number of instances of the sensor.

Methods:

<code>__init__</code> (<code>cfg</code>)	Initializes the ray-caster object.
<code>reset</code> (<code>[env_ids]</code>)	Resets the sensor internals.
<code>set_debug_vis</code> (<code>debug_vis</code>)	Sets whether to visualize the sensor data.

`cfg`: *MultiMeshRayCasterCfg*

The configuration parameters.

`mesh_views`: *ClassVar*[*dict*[*str*, *XformPrimView* | *physx.ArticulationView* | *physx.RigidBodyView*]] = {}

A dictionary to store mesh views for raycasting, shared across all instances.

The keys correspond to the prim path for the mesh views, and values are the corresponding view objects. [Back to top](#)

`__init__`(`cfg`: *MultiMeshRayCasterCfg*)

[\[source\]](#)

Initializes the ray-caster object.

Parameters:

cfg – The configuration parameters.

property data: *MultiMeshRayCasterData*

Data from the sensor.

This property is only updated when the user tries to access the data. This is done to avoid unnecessary computation when the sensor data is not used.

For updating the sensor when this property is accessed, you can use the following code snippet in your sensor implementation:

```
# update sensors if needed
self._update_outdated_buffers()
# return the data (where `_data` is the data for the sensor)
return self._data
```


property device: *str*

Memory device for computation.

property has_debug_vis_implementation: *bool*

Whether the sensor has a debug visualization implemented.

property is_initialized: *bool*

Whether the sensor is initialized.

Returns True if the sensor is initialized, False otherwise.

meshes: *ClassVar[dict[str, wp.Mesh]]* = {}

A dictionary to store warp meshes. [Back to top](#) , casting, shared across all instances.

The keys correspond to the prim path for the meshes, and values are the corresponding warp Mesh objects.

property `num_instances: int`

Number of instances of the sensor.

This is equal to the number of sensors per environment multiplied by the number of environments.

reset(`env_ids: Sequence[int] | None = None`)

Resets the sensor internals.

Parameters:

`env_ids` – The sensor ids to reset. Defaults to None.

set_debug_vis(`debug_vis: bool`) → `bool`

Sets whether to visualize the sensor data.

Parameters:

`debug_vis` – Whether to visualize the sensor data.

Returns:

Whether the debug visualization was successfully set. False if the sensor does not support debug visualization.

class `isaacsim.sensors.MultiMeshRayCasterData` [\[source\]](#)

Data container for the multi-mesh ray-cast sensor.

Attributes:

<code>pos_w</code>	Position of the sensor origin in world frame.
<code>quat_w</code>	Orientation of the sensor origin in quaternion (w, x, y, z) in world frame.
<code>ray_hits_w</code>	The ray hit positions in the world frame. Back to top
<code>ray_mesh_ids</code>	The mesh ids of the ray hits.

pos_w: torch.Tensor = None

Position of the sensor origin in world frame.

Shape is (N, 3), where N is the number of sensors.

quat_w: *torch.Tensor* = *None*

Orientation of the sensor origin in quaternion (w, x, y, z) in world frame.

Shape is (N, 4), where N is the number of sensors.

ray_hits_w: *torch.Tensor* = *None*

The ray hit positions in the world frame.

Shape is (N, B, 3), where N is the number of sensors, B is the number of rays in the scan pattern per sensor.

ray_mesh_ids: *torch.Tensor* = *None*

The mesh ids of the ray hits.

Shape is (N, B, 1), where N is the number of sensors, B is the number of rays in the scan pattern per sensor.

class isaacsim.sensors.MultiMeshRayCasterCfg

[\[source\]](#)

Bases: `RayCasterCfg`

Configuration for the multi-mesh ray-cast sensor.

Classes:

`RaycastTargetCfg`

Configuration for different ray-cast targets.

Attributes:

[Back to top](#)

<code>prim_path</code>	Prim path (or expression) to the sensor.
<code>update_period</code>	Update period of the sensor buffers (in seconds).
<code>history_length</code>	Number of past frames to store in the sensor buffers.
<code>debug_vis</code>	Whether to visualize the sensor.
<code>offset</code>	The offset pose of the sensor's frame from the sensor's parent frame.
<code>attach_yaw_only</code>	Whether the rays' starting positions and directions only track the yaw orientation.
<code>ray_alignment</code>	Specify in what frame the rays are projected onto the ground.
<code>pattern_cfg</code>	The pattern that defines the local ray starting positions and directions.
<code>max_distance</code>	Maximum distance (in meters) from the sensor to ray cast to.
<code>drift_range</code>	The range of drift (in meters) to add to the ray starting positions (xyz) in world frame.
<code>ray_cast_drift_range</code>	The range of drift (in meters) to add to the projected ray points in local projection frame.
<code>visualizer_cfg</code>	The configuration object for the visualization markers.
<code>mesh_prim_paths</code>	The list of mesh primitive paths to ray cast against.
<code>update_mesh_ids</code>	Whether to update the mesh ids of the ray hits in the <code>data</code> container.
<code>reference_meshes</code>	Whether to reference duplicated meshes instead of loading each one separately into memory.

class `RaycastTargetCfg`

[\[source\]](#)

Bases: `object`

[Back to top](#)

Configuration for different ray-cast targets.

Attributes:

<code>prim_expr</code>	The regex to specify the target prim to ray cast against.
<code>is_shared</code>	Whether the target prim is assumed to be the same mesh across all environments.
<code>merge_prim_meshes</code>	Whether to merge the parsed meshes for a prim that contains multiple meshes.
<code>track_mesh_transforms</code>	Whether the mesh transformations should be tracked.

`prim_expr: str`

The regex to specify the target prim to ray cast against.

`is_shared: bool`

Whether the target prim is assumed to be the same mesh across all environments. Defaults to False.

If True, only the first mesh is read and then reused for all environments, rather than re-parsed. This provides a startup performance boost when there are many environments that all use the same asset.

Note

If `MultiMeshRayCasterCfg.reference_meshes` is False, this flag has no effect.

`merge_prim_meshes: bool`

Whether to merge the parsed meshes for a prim that contains multiple meshes. Defaults to True.

This will create a new mesh that combines all meshes in the parsed prim. The raycast hits mesh IDs will then refer to the single merged mesh.

[Back to top](#)

`track_mesh_transforms: bool`

Whether the mesh transformations should be tracked. Defaults to True.

Note

Not tracking the mesh transformations is recommended when the meshes are static to increase performance.

prim_path: *str*

Prim path (or expression) to the sensor.

Note

The expression can contain the environment namespace regex

`{ENV_REGEX_NS}` which will be replaced with the environment namespace.

Example: `{ENV_REGEX_NS}/Robot/sensor` will be replaced with

`/World/envs/env_*/Robot/sensor`.

update_period: *float*

Update period of the sensor buffers (in seconds). Defaults to 0.0 (update every step).

history_length: *int*

Number of past frames to store in the sensor buffers. Defaults to 0, which means that only the current data is stored (no history).

debug_vis: *bool*

Whether to visualize the sensor. Defaults to False.

offset: *OffsetCfg*

The offset pose of the sensor's frame from the sensor's parent frame. Defaults to identity.

[Back to top](#)

attach_yaw_only: *bool* | *None*

Whether the rays' starting positions and directions only track the yaw orientation. Defaults to `None`, which doesn't raise a warning of deprecated usage.

This is useful for ray-casting height maps, where only yaw rotation is needed.

! *Deprecated since version 2.1.1:* This attribute is deprecated and will be removed in the future. Please use `ray_alignment` instead.

To get the same behavior as setting this parameter to `True` or `False`, set `ray_alignment` to `"yaw"` or `"base"` respectively.

`ray_alignment`: *Literal*`['base', 'yaw', 'world']`

Specify in what frame the rays are projected onto the ground. Default is `"base"`.

The options are:

- `base` if the rays' starting positions and directions track the full root position and orientation.
- `yaw` if the rays' starting positions and directions track root position and only yaw component of the orientation. This is useful for ray-casting height maps.
- `world` if rays' starting positions and directions are always fixed. This is useful in combination with a mapping package on the robot and querying ray-casts in a global frame.

`pattern_cfg`: *PatternBaseCfg*

The pattern that defines the local ray starting positions and directions.

`max_distance`: *float*

Maximum distance (in meters) from the sensor to ray cast to. Defaults to `1e6`.

[Back to top](#)

`drift_range`: *tuple*`[float, float]`

The range of drift (in meters) to add to the ray starting positions (xyz) in world frame. Defaults to `(0.0, 0.0)`.

For floating base robots, this is useful for simulating drift in the robot's pose estimation.

ray_cast_drift_range: *dict[str, tuple[float, float]]*

The range of drift (in meters) to add to the projected ray points in local projection frame. Defaults to a dictionary with zero drift for each x, y and z axis.

For floating base robots, this is useful for simulating drift in the robot's pose estimation.

visualizer_cfg: *VisualizationMarkersCfg*

The configuration object for the visualization markers. Defaults to RAY_CASTER_MARKER_CFG.

Note

This attribute is only used when debug visualization is enabled.

mesh_prim_paths: *list[str | RaycastTargetCfg]*

The list of mesh primitive paths to ray cast against.

If an entry is a string, it is internally converted to `RaycastTargetCfg` with `track_mesh_transforms` disabled. These settings ensure backwards compatibility with the default raycaster.

update_mesh_ids: *bool*

Whether to update the mesh ids of the ray hits in the `data` container.

reference_meshes: *bool*

Whether to reference duplicated meshes instead of loading each one separately into memory. Defaults to True.

[Back to top](#)

When enabled, the raycaster parses all meshes in all environments, but reuses references for duplicates instead of storing multiple copies. This reduces memory footprint.

Multi-Mesh Ray-Cast Camera

`class isaacsim.sensors.MultiMeshRayCasterCamera`

[\[source\]](#)

Bases: `RayCasterCamera`, `MultiMeshRayCaster`

A multi-mesh ray-casting camera sensor.

The ray-caster camera uses a set of rays to get the distances to meshes in the scene. The rays are defined in the sensor's local coordinate frame. The sensor has the same interface as the `isaacsim.sensors.Camera` that implements the camera class through USD camera prims. However, this class provides a faster image generation. The sensor converts meshes from the list of primitive paths provided in the configuration to Warp meshes. The camera then ray-casts against these Warp meshes only.

Currently, only the following annotators are supported:

- `"distance_to_camera"`: An image containing the distance to camera optical center.
- `"distance_to_image_plane"`: An image containing distances of 3D points from camera plane along camera's z-axis.
- `"normals"`: An image containing the local surface normal vectors at each pixel.

Attributes:

[Back to top](#)

<code>cfg</code>	The configuration parameters.
<code>UNSUPPORTED_TYPES</code>	A set of sensor types that are not supported by the ray-caster camera.
<code>data</code>	Data from the sensor.
<code>device</code>	Memory device for computation.
<code>frame</code>	Frame number when the measurement took place.
<code>has_debug_vis_implementation</code>	Whether the sensor has a debug visualization implemented.
<code>image_shape</code>	A tuple containing (height, width) of the camera sensor.
<code>is_initialized</code>	Whether the sensor is initialized.
<code>mesh_views</code>	A dictionary to store mesh views for raycasting, shared across all instances.
<code>meshes</code>	A dictionary to store warp meshes for raycasting, shared across all instances.
<code>num_instances</code>	Number of instances of the sensor.

Methods:

<code>__init__</code> (cfg)	Initializes the camera object.
<code>reset</code> ([env_ids])	Resets the sensor internals.
<code>set_debug_vis</code> (debug_vis)	Sets whether to visualize the sensor data.
<code>set_intrinsic_matrices</code> (matrices[, ...])	Set the intrinsic matrix of the camera.
<code>set_world_poses</code> ([positions, c Back to top .])	Set the pose of the camera w.r.t.
<code>set_world_poses_from_view</code> (eyes, targets[, ...])	Set the poses of the camera from the eye position and look-at target position.

cfg: *MultiMeshRayCasterCameraCfg*

The configuration parameters.

`__init__`(cfg: *MultiMeshRayCasterCameraCfg*)
[\[source\]](#)

Initializes the camera object.

Parameters:

cfg – The configuration parameters.

Raises:

ValueError – If the provided data types are not supported by the ray-caster camera.

```
UNSUPPORTED_TYPES: ClassVar[set[str]] = {'bounding_box_2d_loose',
'bounding_box_2d_loose_fast', 'bounding_box_2d_tight',
'bounding_box_2d_tight_fast', 'bounding_box_3d',
'bounding_box_3d_fast', 'instance_id_segmentation',
'instance_id_segmentation_fast', 'instance_segmentation',
'instance_segmentation_fast', 'motion_vectors', 'rgb',
'semantic_segmentation', 'skeleton_data'}
```

A set of sensor types that are not supported by the ray-caster camera.

property data: *CameraData*

Data from the sensor.

This property is only updated when the user tries to access the data. This is done to avoid unnecessary computation when the sensor data is not used.

For updating the sensor when this property is accessed, you can use the following code snippet in your sensor implementation:

```
# update sensors if ne    Back to top
self._update_outdated_buffers()
# return the data (where `_data` is the data for the sensor)
return self._data
```



property **device**: *str*

Memory device for computation.

property **frame**: *torch.tensor*

Frame number when the measurement took place.

property **has_debug_vis_implementation**: *bool*

Whether the sensor has a debug visualization implemented.

property **image_shape**: *tuple[int, int]*

A tuple containing (height, width) of the camera sensor.

property **is_initialized**: *bool*

Whether the sensor is initialized.

Returns True if the sensor is initialized, False otherwise.

mesh_views: *ClassVar[dict[str, XformPrimView | physx.ArticulationView | physx.RigidBodyView]] = {}*

A dictionary to store mesh views for raycasting, shared across all instances.

The keys correspond to the prim path for the mesh views, and values are the corresponding view objects.

meshes: *ClassVar[dict[str, wp.Mesh]] = {}*

A dictionary to store warp meshes for raycasting, shared across all instances.

The keys correspond to the [Back to top](#) the meshes, and values are the corresponding warp Mesh objects.

property **num_instances**: *int*

Number of instances of the sensor.

This is equal to the number of sensors per environment multiplied by the number of environments.

reset(*env_ids*: *Sequence[int] | None = None*)

Resets the sensor internals.

Parameters:

env_ids – The sensor ids to reset. Defaults to None.

set_debug_vis(*debug_vis*: *bool*) → *bool*

Sets whether to visualize the sensor data.

Parameters:

debug_vis – Whether to visualize the sensor data.

Returns:

Whether the debug visualization was successfully set. False if the sensor does not support debug visualization.

set_intrinsic_matrices(*matrices*: *torch.Tensor*, *focal_length*: *float* = 1.0, *env_ids*: *Sequence[int] | None = None*)

Set the intrinsic matrix of the camera.

Parameters:

- **matrices** – The intrinsic matrices for the camera. Shape is (N, 3, 3).
- **focal_length** – Focal length to use when computing aperture values (in cm). Defaults to 1.0.
- **env_ids** – A sensor ids to manipulate. Defaults to None, which means all sensor indices.

[Back to top](#)

set_world_poses(*positions*: *torch.Tensor* | *None* = None, *orientations*: *torch.Tensor* | *None* = None, *env_ids*: *Sequence[int] |*

None = None, **convention**: *Literal*['opengl', 'ros', 'world'] = 'ros')

Set the pose of the camera w.r.t. the world frame using specified convention.

Since different fields use different conventions for camera orientations, the method allows users to set the camera poses in the specified convention. Possible conventions are:

- **"opengl"** - forward axis: -Z - up axis +Y - Offset is applied in the OpenGL (Usd.Camera) convention
- **"ros"** - forward axis: +Z - up axis -Y - Offset is applied in the ROS convention
- **"world"** - forward axis: +X - up axis +Z - Offset is applied in the World Frame convention

See `isaacsim.utils.maths.convert_camera_frame_orientation_convention()` for more details on the conventions.

Parameters:

- **positions** – The cartesian coordinates (in meters). Shape is (N, 3). Defaults to None, in which case the camera position is not changed.
- **orientations** – The quaternion orientation in (w, x, y, z). Shape is (N, 4). Defaults to None, in which case the camera orientation is not changed.
- **env_ids** – A sensor ids to manipulate. Defaults to None, which means all sensor indices.
- **convention** – The convention in which the poses are fed. Defaults to "ros".

Raises:

RuntimeError – If the camera prim is not set. Need to call `initialize()` method first.

set_world_poses_from_view(eyes: *torch.Tensor*, targets: *torch.Tensor*, env_ids: *Sequence*[int] | None = None)

Set the poses of the camera from the eye position and look-at target position.

Parameters:

[Back to top](#)

- **eyes** – The positions of the camera's eye. Shape is $N, 3$).
- **targets** – The target locations to look at. Shape is $(N, 3)$.
- **env_ids** – A sensor ids to manipulate. Defaults to `None`, which means all sensor indices.

Raises:

- **RuntimeError** – If the camera prim is not set. Need to call `initialize()` method first.
- **NotImplementedError** – If the stage up-axis is not “Y” or “Z”.

`class isaacsim.sensors.MultiMeshRayCasterCameraCfg` [\[source\]](#)

Bases: `RayCasterCameraCfg`, `MultiMeshRayCasterCfg`

Configuration for the multi-mesh ray-cast camera sensor.

Attributes:

[Back to top](#)

<code>prim_path</code>	Prim path (or expression) to the sensor.
<code>update_period</code>	Update period of the sensor buffers (in seconds).
<code>history_length</code>	Number of past frames to store in the sensor buffers.
<code>debug_vis</code>	Whether to visualize the sensor.
<code>mesh_prim_paths</code>	The list of mesh primitive paths to ray cast against.
<code>offset</code>	The offset pose of the sensor's frame from the sensor's parent frame.
<code>attach_yaw_only</code>	Whether the rays' starting positions and directions only track the yaw orientation.
<code>ray_alignment</code>	Specify in what frame the rays are projected onto the ground.
<code>pattern_cfg</code>	The pattern that defines the local ray starting positions and directions in a pinhole camera pattern.
<code>max_distance</code>	Maximum distance (in meters) from the sensor to ray cast to.
<code>drift_range</code>	The range of drift (in meters) to add to the ray starting positions (xyz) in world frame.
<code>ray_cast_drift_range</code>	The range of drift (in meters) to add to the projected ray points in local projection frame.
<code>visualizer_cfg</code>	The configuration object for the visualization markers.
<code>update_mesh_ids</code>	Whether to update the mesh ids of the ray hits in the <code>data</code> container.
<code>reference_meshes</code>	Whether to reference duplicated meshes instead of loading each one separately into memory. Back to top
<code>data_types</code>	List of sensor names/types to enable for the camera.
<code>depth_clipping_behavior</code>	Clipping behavior for the camera for values exceed the maximum value.

prim_path: *str*

Prim path (or expression) to the sensor.

Note

The expression can contain the environment namespace regex

`{ENV_REGEX_NS}` which will be replaced with the environment namespace.

Example: `{ENV_REGEX_NS}/Robot/sensor` will be replaced with

`/World/envs/env_.* /Robot/sensor`.

update_period: *float*

Update period of the sensor buffers (in seconds). Defaults to 0.0 (update every step).

history_length: *int*

Number of past frames to store in the sensor buffers. Defaults to 0, which means that only the current data is stored (no history).

debug_vis: *bool*

Whether to visualize the sensor. Defaults to False.

mesh_prim_paths: *list[str]*

The list of mesh primitive paths to ray cast against.

If an entry is a string, it is internally converted to `RaycastTargetCfg` with

`track_mesh_transforms` disabled. These settings ensure backwards compatibility with the default raycaster.

offset: *OffsetCfg*

[Back to top](#)

The offset pose of the sensor's frame from the sensor's parent frame. Defaults to identity.

attach_yaw_only: *bool* | *None*

Whether the rays' starting positions and directions only track the yaw orientation. Defaults to *None*, which doesn't raise a warning of deprecated usage.

This is useful for ray-casting height maps, where only yaw rotation is needed.

! *Deprecated since version 2.1.1:* This attribute is deprecated and will be removed in the future. Please use `ray_alignment` instead.

To get the same behavior as setting this parameter to `True` or `False`, set `ray_alignment` to `"yaw"` or `"base"` respectively.

ray_alignment: *Literal*['base', 'yaw', 'world']

Specify in what frame the rays are projected onto the ground. Default is `"base"`.

The options are:

- `base` if the rays' starting positions and directions track the full root position and orientation.
- `yaw` if the rays' starting positions and directions track root position and only yaw component of the orientation. This is useful for ray-casting height maps.
- `world` if rays' starting positions and directions are always fixed. This is useful in combination with a mapping package on the robot and querying ray-casts in a global frame.

pattern_cfg: *PinholeCameraPatternCfg*

The pattern that defines the local ray starting positions and directions in a pinhole camera pattern.

max_distance: *float*

Maximum distance (in meters) for sensor to ray cast to. Defaults to 1e6.

drift_range: *tuple*[*float*, *float*]

The range of drift (in meters) to add to the ray starting positions (xyz) in world frame.

Defaults to (0.0, 0.0).

For floating base robots, this is useful for simulating drift in the robot's pose estimation.

ray_cast_drift_range: *dict[str, tuple[float, float]]*

The range of drift (in meters) to add to the projected ray points in local projection frame. Defaults to a dictionary with zero drift for each x, y and z axis.

For floating base robots, this is useful for simulating drift in the robot's pose estimation.

visualizer_cfg: *VisualizationMarkersCfg*

The configuration object for the visualization markers. Defaults to RAY_CASTER_MARKER_CFG.

Note

This attribute is only used when debug visualization is enabled.

update_mesh_ids: *bool*

Whether to update the mesh ids of the ray hits in the `data` container.

reference_meshes: *bool*

Whether to reference duplicated meshes instead of loading each one separately into memory. Defaults to True.

When enabled, the raycaster parses all meshes in all environments, but reuses references for duplicates instead of storing multiple copies. This reduces memory footprint.

data_types: *list[str]*

[Back to top](#)

List of sensor names/types to enable for the camera. Defaults to ["distance_to_image_plane"].

depth_clipping_behavior: *Literal*['max', 'zero', 'none']

Clipping behavior for the camera for values exceed the maximum value. Defaults to “none”.

- `"max"`: Values are clipped to the maximum value.
- `"zero"`: Values are clipped to zero.
- `"none"`: No clipping is applied. Values will be returned as `inf` for `distance_to_camera` and `nan` for `distance_to_image_plane` data type.

Inertia Measurement Unit

`class isaacsim.sensors.Imu`

[\[source\]](#)

Bases: `SensorBase`

The Inertia Measurement Unit (IMU) sensor.

The sensor can be attached to any prim path with a rigid ancestor in its tree and produces body-frame linear acceleration and angular velocity, along with world-frame pose and body-frame linear and angular accelerations/velocities.

If the provided path is not a rigid body, the closest rigid-body ancestor is used for simulation queries. The fixed transform from that ancestor to the target prim is computed once during initialization and composed with the configured sensor offset.

Note

We are computing the accelerations using numerical differentiation from the velocities. Consequently, the IMU sensor accuracy depends on the chosen physics timestep. For a sufficient accuracy, we recommend to keep the timestep at least as 200Hz.

[Back to top](#)

Note

The user can configure the sensor offset in the configuration file. The offset is applied relative to the rigid source prim. If the target prim is not a rigid body, the offset is composed with the fixed transform from the rigid ancestor to the target prim. The offset is applied in the body frame of the rigid source prim. The offset is defined as a position vector and a quaternion rotation, which are applied in the order: position, then rotation. The position is applied as a translation in the body frame of the rigid source prim, and the rotation is applied as a rotation in the body frame of the rigid source prim.

Attributes:

<code>cfg</code>	The configuration parameters.
<code>data</code>	Data from the sensor.
<code>num_instances</code>	Number of instances of the sensor.
<code>device</code>	Memory device for computation.
<code>has_debug_vis_implementation</code>	Whether the sensor has a debug visualization implemented.
<code>is_initialized</code>	Whether the sensor is initialized.

Methods:

<code>__init__</code> (<code>cfg</code>)	Initializes the Imu sensor.
<code>reset</code> (<code>[env_ids]</code>)	Resets the sensor internals.
<code>set_debug_vis</code> (<code>debug_vis</code>)	Sets whether to visualize the sensor data.

cfg: *ImuCfg*

The configuration paramete

[Back to top](#)

`__init__`(`cfg: ImuCfg`)

[\[source\]](#)

Initializes the Imu sensor.

Parameters:

cfg – The configuration parameters.

property data: ImuData

Data from the sensor.

This property is only updated when the user tries to access the data. This is done to avoid unnecessary computation when the sensor data is not used.

For updating the sensor when this property is accessed, you can use the following code snippet in your sensor implementation:

```
# update sensors if needed
self._update_outdated_buffers()
# return the data (where `_data` is the data for the sensor)
return self._data
```

**property num_instances: int**

Number of instances of the sensor.

This is equal to the number of sensors per environment multiplied by the number of environments.

reset(env_ids: Sequence[int] | None = None)[\[source\]](#)

Resets the sensor internals.

Parameters:

env_ids – The sensor ids to reset. Defaults to None.

property device: str

Memory device for computa. [Back to top](#)

property has_debug_vis_implementation: bool

Whether the sensor has a debug visualization implemented.

property `is_initialized: bool`

Whether the sensor is initialized.

Returns True if the sensor is initialized, False otherwise.

set_debug_vis(`debug_vis: bool`) → `bool`

Sets whether to visualize the sensor data.

Parameters:

`debug_vis` – Whether to visualize the sensor data.

Returns:

Whether the debug visualization was successfully set. False if the sensor does not support debug visualization.

class `isaacsim.sensors.ImuCfg`

[\[source\]](#)

Bases: `SensorBaseCfg`

Configuration for an Inertial Measurement Unit (IMU) sensor.

Classes:

`OffsetCfg`

The offset pose of the sensor's frame from the sensor's parent frame.

Attributes:

[Back to top](#)

<code>prim_path</code>	Prim path (or expression) to the sensor.
<code>update_period</code>	Update period of the sensor buffers (in seconds).
<code>history_length</code>	Number of past frames to store in the sensor buffers.
<code>debug_vis</code>	Whether to visualize the sensor.
<code>offset</code>	The offset pose of the sensor's frame from the sensor's parent frame.
<code>visualizer_cfg</code>	The configuration object for the visualization markers.
<code>gravity_bias</code>	The linear acceleration bias applied to the linear acceleration in the world frame (x,y,z).

class `OffsetCfg`

[\[source\]](#)

Bases: `object`

The offset pose of the sensor's frame from the sensor's parent frame.

Attributes:

<code>pos</code>	Translation w.r.t.
<code>rot</code>	Quaternion rotation (w, x, y, z) w.r.t.

pos: `tuple[float, float, float]`

Translation w.r.t. the parent frame. Defaults to (0.0, 0.0, 0.0).

rot: `tuple[float, float, float, float]`

Quaternion rotation (w, x, y, z) w.r.t. the parent frame. Defaults to (1.0, 0.0, 0.0, 0.0).

prim_path: `str`

Prim path (or expression) to

[Back to top](#)

Note

The expression can contain the environment namespace regex

`{ENV_REGEX_NS}` which will be replaced with the environment namespace.

Example: `{ENV_REGEX_NS}/Robot/sensor` will be replaced with

`/World/envs/env_.* /Robot/sensor`.

update_period: *float*

Update period of the sensor buffers (in seconds). Defaults to 0.0 (update every step).

history_length: *int*

Number of past frames to store in the sensor buffers. Defaults to 0, which means that only the current data is stored (no history).

debug_vis: *bool*

Whether to visualize the sensor. Defaults to False.

offset: *OffsetCfg*

The offset pose of the sensor's frame from the sensor's parent frame. Defaults to identity.

visualizer_cfg: *VisualizationMarkersCfg*

The configuration object for the visualization markers. Defaults to RED_ARROW_X_MARKER_CFG.

This attribute is only used when debug visualization is enabled.

[Back to top](#)

gravity_bias: *tuple[float, float, float]*

The linear acceleration bias applied to the linear acceleration in the world frame (x,y,z).

Imu sensors typically output a positive gravity acceleration in opposition to the direction

of gravity. This config parameter allows users to subtract that bias if set to (0.,0.,0.). By default this is set to (0.0,0.0,9.81) which results in a positive acceleration reading in the world Z.

[Back to top](#)